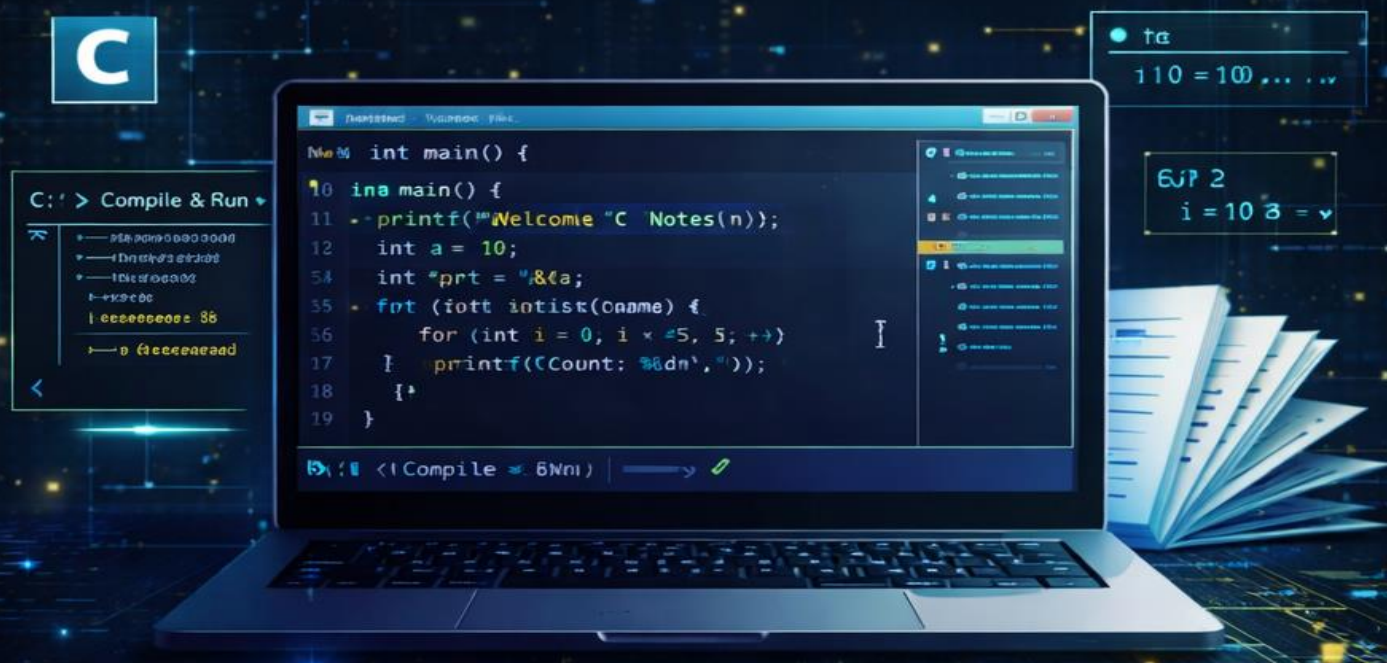
C Language

Complete Notes

Learn C programming from basics to advanced
with real-world examples



Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

Contents

✅ C LANGUAGE BASICS 6

C Language Introduction –..... 6

C Language Se Kya-Kya Bana Sakte Ho? (Real World Uses) 6

C Sikhne Ke Baad Python, Java, JavaScript Kyun Bahut Easy Lagte Hain? 7

Sabse Pehla Program – Hello World (Line by Line Samjhaata Hoon) 7

printf aur \n Ka Khel (Bahut Important) 8

✅ Conio.h 8

conio.h Kya Hai? (Sabse Simple) 8

conio.h Mein Sabse Important Functions (Pura List + Example) 9

3. Poora Example Program (Sab Functions Ek Saath) 10

Limitations (Important Baat) 10

✅ Stdio.h 11

stdio.h Kya Hai? 11

stdio.h Mein Sabse Important Functions (Pura List + Example) 11

Poora Practical Example (Sab Ek Saath) 12

stdio.h vs conio.h (Quick Comparison) 13

✅ VARIABLE 13

Variable Kya Hai? (Sabse Simple Explanation) 13

Variable Naming Rules (Yeh Rules Yaad Rakhna Zaroori Hai) 13

✅ DATA TYPES IN C 14

Declaration aur Initialization 14

Complete Practical Example 14

Common Mistakes Beginners Karte Hain 16

✅ PRACTICE SET 1 16

Section 1: Variables & Data Types (10 Questions) 16

Section 2: stdio.h (printf & scanf) (8 Questions) 17

Section 3: conio.h (clrscr, getch, gotoxy etc.) (7 Questions) 18

✅ INSTRUCTIONS (STATEMENTS) AUR ✅ OPERATORS IN C 19

1. Instructions (Statements) Kya Hote Hain? 19

2. Operators in C (Pura Detail) 19

C Language Complete Notes
Learn C programming from basics to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

3. Operator Precedence (Hierarchy) – Sabse Important 20

✅ PRACTICE SET 2 21

✅ CONDITIONAL STATEMENTS 25

1. if Statement (Sabse Basic) 25

2. if-else Statement 25

3. else-if Ladder (Multiple Conditions) 26

4. switch-case Statement 26

✅ PRACTICE SET 3 27

✅ LOOP CONTROL STATEMENTS 32

1. for Loop (Sabse Zyada Use Hota Hai) 32

2. while Loop 33

3. do-while Loop (Kam se kam ek baar toh chalega) 33

4. Loop Control Keywords 34

✅ PRACTICE SET 4 34

✅ FUNCTIONS 36

1. Functions Kya Hote Hain? 36

2. Function Ka Structure 36

3. Practical Examples of Functions 36

4. Types of Functions 39

5. Recursion Kya Hai? 39

6. Practical Examples of Recursion 39

✅ PRACTICE SET 5 40

✅ POINTERS 45

1. Pointer Kya Hai? (Sabse Simple Explanation) 45

2. Pointer Kyun Zaroori Hai? 45

3. Pointer Ka Syntax (Declaration) 45

4. Important Operators 46

5. Pointers ke Practical Examples 46

6. Pointers and Arrays (Sabse Important) 47

7. Pointers and Functions (Call by Reference) 47

8. NULL Pointer 47

✅ PRACTICE SET 6 48
✅ ARRAYS IN C 52
1. Array Kya Hai? 52
2. Array Kyun Use Karte Hain? 52
3. 1D Array (One-Dimensional Array) 52
4. Array Elements Access Karna 53
5. Memory Representation 53
6. Loops ke Saath Array 53
7. Passing Array to Function 53
8. Pointers aur Arrays ka Relation (Bahut Important) 54
9. 2D Arrays (Two-Dimensional Arrays) 54
✅ PRACTICE SET 7 55
✅ STRINGS IN C 59
1. String Kya Hai? 59
2. String Declaration & Initialization 59
3. String Input & Output 59
4. <string.h> Header Functions (Sabse Important) 60
5. Practical Examples (String Functions) 60
6. Manual Implementation (Without string.h) 61
7. 2D Array of Strings 61
8. Pointers with Strings 62
✅ PRACTICE SET 8 62
✅ STRUCTURES IN C 69
1. Structure Kya Hai? 70
2. Syntax of Structure 70
3. Structure Variable Create Karna 70
4. Accessing Structure Members (Dot Operator .) 71
5. Initialization of Structure 72
6. Array of Structures 72
7. Pointer to Structure 72
8. Nested Structure (Structure inside Structure) 72

C Language
Complete Notes
Learn C programming from basics to advanced
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

9. Passing Structure to Function 73
10. typedef with Structure (Code Short Karne Ke Liye) 73
✅ PRACTICE SET 9 74
✅ FILE I/O 10 82
1. File I/O Kya Hai? 82
2. File Handling Ke 3 Basic Steps 82
3. File Modes (Sabse Important) 82
4. File Pointer Kya Hai? 83
5. Step-by-Step Practical Example 83
6. File Se Data Padhna (Reading) 84
7. File Mein Data Likhna (Writing) 84
8. File Se Data Padhna (Reading) 85
9. File Mein Naya Data Add Karna (Append Mode) 85
✅ PRACTICE SET 11 86
✅ DYNAMIC MEMORY ALLOCATION IN C – 92
1. Dynamic Memory Allocation Kya Hai? 92
2. Kyun Zarurat Padti Hai? 92
3. 4 Important Functions (Sabse Important) 92
4. Detailed Explanation with Examples 93
5. Common Mistakes (Yaad Rakhna) 94
✅ PRACTICE SET 12 94
✅ MIXED PRACTICAL QUESTIONS 100

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ C LANGUAGE BASICS

C Language Introduction –

C ek low-level programming language hai. Ise **Dennis Ritchie** ne 1972 mein Bell Labs (USA) mein banaya tha. Yeh **procedural language** hai matlab step-by-step instructions likhte ho.

C Ki Special Features (Sabse Important):

- Bahut **fast** aur powerful (machine ke close hai)
- **Memory** ko direct control milta hai (pointers ke through)
- **Portable** – ek baar code likho, kahi bhi chala sakte ho
- Operating System, hardware, embedded devices ke liye best
- Modern languages (Python, Java, C++, JavaScript) ka **baap** mana jata hai

Introduction Course Mein Yeh Topics Cover Hote Hain (Sequence):

Topic	Kya Sikhoge?	Kitna Important
History & Features	Dennis Ritchie, C ka birth story	Basic
Data Types & Variables	int, float, char, double	★★★★★
Operators	Arithmetic, Logical, Bitwise	★★★★
Input/Output (printf, scanf)	Console pe input-output	★★★★★
Control Statements	if-else, switch, loops (for, while, do-while)	★★★★★
Functions	User-defined & Library functions	★★★★★
Arrays & Strings	1D, 2D array, string handling	★★★★
Pointers	Memory address, pointer arithmetic	★★★★★ (Sabse Hard)
Structures & Unions	Custom data types	★★★
File Handling	File read/write	★★★
Dynamic Memory (malloc)	Runtime memory allocation	Advanced

Yeh sab seekh lene ke baad C ka basic **strong foundation** ban jata hai.

C Language Se Kya-Kya Bana Sakte Ho? (Real World Uses)

C sirf "hello world" nahi hai – yeh **real powerful software** banane ke liye use hota hai:

Real Big Things Jo C Se Bane Hain:

- **Operating Systems** → Linux kernel, Windows ke kuch parts, Unix
- **Embedded Systems** → Microwave, Washing machine, Car ke ECU, Drones
- **IoT Devices** → Smart bulbs, sensors, Arduino projects
- **Game Engines** → Unreal Engine ke core parts (C++ mein bhi C style use hota hai)
- **Compilers & Interpreters** → Python interpreter bhi C mein likha gaya hai
- **Databases** → MySQL, PostgreSQL ke core parts
- **Browsers** → Google Chrome ke kuch low-level parts
- **Antivirus, Drivers, Firmware**

Beginner Se Advanced Projects (Tu Khud Bana Sakta Hai):

Level	Project Examples
Beginner	Calculator, Number guessing game, Simple ATM
Intermediate	Tic-Tac-Toe, Snake game, Library Management System
Advanced	Mini Operating System, Text Editor, Chat Application, Mini Compiler

C Sikhne Ke Baad Python, Java, JavaScript Kyun Bahut Easy Lagte Hain?

Yeh sabse important point hai jo log miss kar dete hain.

C Sikhne Ke Fayde (Python/Java ko Asaan Kyun Bana Deta Hai):

1. **Memory Management Samajh Aata Hai** Python mein garbage collector chalta hai, lekin C mein tum khud malloc aur free karte ho → Python ke andar kya chal raha hai samajh mein aata hai.
2. **Pointers & Low-Level Concepts Clear Ho Jaate Hain** Pointers sabse mushkil part hota hai C ka, lekin seekh lene ke baad JavaScript ke references, Python ke objects sab easy lagte hain.
3. **Logic & Problem Solving Bahut Strong Ho Jati Hai** C mein har chhoti-chhoti cheez manually likhni padti hai → isliye logic bahut tight ho jati hai.
4. **Error Debugging Ka Funda Strong** C mein error milta hai toh khud solve karna padta hai → Python mein error aaye toh 2 minute mein fix kar loge.
5. **Performance Ka Concept Clear** Python slow kyun hai? Kyunki woh C pe based hai. C seekhne ke baad tumhe pata chalega ki Python ke andar kya ho raha hai.

Sabse Pehla Program – Hello World (Line by Line Samjhaata Hoon)

Yeh sabse basic program hai jo har beginner likhta hai:

C

```c
#include <stdio.h>

int main()
{
    printf("Hello World!\n");
    return 0;
}
```

1. **#include <stdio.h>**
   a. #include ka matlab = "include kar lo" (kuch extra facilities laao).
   b. stdio.h = Standard Input Output header file.
   c. Iska matlab: Mujhe **print karne** aur **input lene** ki permission do.
   d. Yeh line har C program mein almost hamesha likhi jaati hai.
2. **int main()**
   a. main = yahin se program shuru hota hai (jaise movie ka starting scene).
   b. int = integer (return value integer hoga).
   c. () = khali brackets, abhi ke liye ignore kar.
3. **{ … }**
   a. Curly brackets ke andar saara code likhte hain jo main function mein chalega.
4. **printf("Hello World!\n");**

Complete Notes
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

a. printf = print function (screen pe likhne ke liye).
b. Double quotes (" ") ke andar jo likha hai, woh print hota hai.
c. \n = new line (enter mar do, agle line mein jaao).
d. ; = statement khatam ho gaya (har line ke end mein lagana zaroori hai).

5. **return 0;**
   a. Program successfully khatam ho gaya.
   b. 0 ka matlab = sab theek hai.

Output:

[redacted]

printf aur \n Ka Khel (Bahut Important)

printf se hum kuch bhi screen pe likh sakte hain.

Example 1:

C

```
printf("Hello PW");
printf("Hello College");
```

Output: Hello PWHello College (ek hi line mein)

Example 2 (with \n):

C

```
printf("Hello PW\n");
printf("Hello College");
```

Output:

```text
Hello PW
Hello College
```

Ek line mein multiple \n:

C

```
printf("Hello\nPW\nCollege\n");
```

Output:

```text
Hello
PW
College
```

✅ Conio.h

conio.h Kya Hai? (Sabse Simple)

- conio.h = **Console Input Output Header File**
- Yeh C language ka ek **header file** hai jo **screen (console) ko direct control** karne ke liye use hota hai.
- Iska matlab: Normal printf aur scanf se zyada powerful cheezein kar sakte ho jaise:
  - Screen saaf karna

- Cursor ko kahin bhi le jaana
- Keyboard se ek character turant lena (Enter dabane ki zarurat nahi)
- Text ka color change karna
- Screen par direct character print karna

Important Note:

- Yeh **standard C** ka hissa **nahi** hai (ANSI C / ISO C mein nahi hai).
- Yeh sirf **Turbo C / Turbo C++ / Borland C** jaise purane compilers mein achhe se kaam karta hai.
- Modern compilers (GCC, VS Code, online compiler) mein yeh **nahi** chalta ya warning/error deta hai.

conio.h Mein Sabse Important Functions (Pura List + Example)

Yeh sab functions conio.h include karne ke baad use kar sakte ho:

C

```
#include <conio.h>      // ← Yeh line sabse upar likho
```

1. clrscr() → Screen Clear Karne Ke Liye

C

```
clrscr();               // pura screen saaf ho jayega
printf("Hello");
```

getch() → Ek Character Lene Ke Liye (Sabse Famous)

- Enter dabane ki zarurat nahi
- Jo character dabaya, woh screen par nahi dikhega (no echo)

Example:

C

```
char ch;
printf("Koi key dabao: ");
ch = getch();           // turant character le lega
printf("\nTumne yeh dabaya: %c", ch);
```

getche() → Character Lene Ke Liye (Echo ke saath)

- getch() jaisa hi, lekin jo character dabaya woh screen par dikh jaayega.

4. putch() → Sirf Ek Character Print Karne Ke Liye

C

```
putch('A');     // sirf A print hoga
```

5. gotoxy(x, y) → Cursor Ko Screen Par Kahin Bhi Le Jaane Ke Liye

- x = column (left to right)
- y = row (top to bottom)

Example:

C

```
gotoxy(10, 5);     // 10th column, 5th row par cursor le jaayega
printf("Yahan likh raha hoon");
```

textcolor() aur textbackground() → Color Change

C

C Language Complete Notes
Learn C programming from basics to advanced with real-world examples
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
textcolor(RED);            // text ka color red
textbackground(BLUE);      // background blue
cprintf("Colored Text");   // normal printf nahi, cprintf use karo
```

Common Colors: BLACK, BLUE, GREEN, CYAN, RED, MAGENTA, YELLOW, WHITE

cprintf() → Colored Printf

Normal printf ki jagah cprintf use karo color ke saath.

kbhit() → Check Karo Ki Koi Key Press Hui Hai Ya Nahi

C

```
while(!kbhit()) {     // jab tak key nahi dabai, loop chalega
    printf(".");
}
```

delay(ms) → Kitne millisecond wait karna hai (#include <dos.h>)

C

```
delay(1000);     // 1 second wait (1000 ms = 1 second)
```

3. Poora Example Program (Sab Functions Ek Saath)

C

```
#include <stdio.h>
#include <conio.h>

int main()
{
    clrscr();                    // screen saaf
    gotoxy(20, 5);
    textcolor(YELLOW);
    cprintf("Hello! Ye conio.h ka example hai\n");

    printf("Koi key dabao...");
    getch();                     // wait for key press

    clrscr();
    printf("Ab screen saaf ho gayi!");

    getch();                     // program rokne ke liye
    return 0;
}
```

Limitations (Important Baat)

- **Modern compilers** (GCC, VS Code, Code::Blocks, onlinegdb) mein **nahi** chalta.
- Agar try karoge toh error aayega: fatal error: conio.h: No such file or directory
- Sirf **Turbo C** mein best kaam karta hai (college labs mein zyadatar yahi use hota hai).
- Aaj kal log iski jagah **standard functions** use karte hain (jaise system("cls") Windows ke liye).

C Language Complete Notes with real-world examples
THE ARTIFICIAL EDUCATION

✅ Stdio.h

stdio.h Kya Hai?

- **stdio.h = Standard Input Output Header File**
- Yeh C ka **sabse important aur sabse common** header file hai.
- Iska kaam: **Input lena** (keyboard se) aur **Output dena** (screen pe).
- Har C program mein almost hamesha yeh include karna padta hai.

C

```
#include <stdio.h>      // ← Yeh line har program ke sabse upar likho
```

Agar yeh line nahi likhoge toh printf, scanf wagairah kaam nahi karenge.

stdio.h Mein Sabse Important Functions (Pura List + Example)

1. printf() → Screen pe kuch bhi print karne ke liye

Sabse zyada use hone wala function.

C

```
printf("Hello World\n");
printf("Value = %d\n", 25);
printf("Name = %s aur Age = %d\n", "Pawan", 22);
```

Format Specifiers (Yaad rakhne wale):

- %d → int (pura number)
- %f → float (decimal)
- %c → char (single character)
- %s → string (text)

2. scanf() → User se input lene ke liye

C

```
int age;
scanf("%d", &age);            // & lagana zaroori hai

float marks;
scanf("%f", &marks);

char ch;
scanf(" %c", &ch);            // char ke saath space daalna achha hota hai
```

Bahut Important Rule: Har variable ke saath & (ampersand) lagana padta hai. Ye address deta hai.

Multiple values ek saath:

C

```
int a, b;
scanf("%d %d", &a, &b);
```

3. getchar() → Ek character input lene ke liye

C

THE ARTIFICIAL EDUCATION

```
char ch = getchar();
printf("Tumne dabaya: %c", ch);
```

4. putchar() → Ek character print karne ke liye

C

```
putchar('A');
putchar(ch);
```

5. gets() aur puts() (purane, lekin beginner ke liye)

- gets(str) → pura line (string) input lene ke liye
- puts(str) → pura string print karne ke liye

Note: Aaj kal gets() ko unsafe mana jaata hai, isliye fgets() use karte hain (advanced level).

6. File handling functions (thoda advanced)

- fopen(), fclose(), fprintf(), fscanf(), fread(), fwrite() etc.

Poora Practical Example (Sab Ek Saath)

C

```
#include <stdio.h>

int main()
{
    int age;
    float marks;
    char grade;
    char name[50];

    printf("Apna naam batao: ");
    scanf("%s", name);                    // string ke liye & nahi lagate

    printf("Apni age batao: ");
    scanf("%d", &age);

    printf("Apne marks batao: ");
    scanf("%f", &marks);

    printf("Apna grade batao (A/B/C): ");
    scanf(" %c", &grade);                 // space daala hai

    printf("\n--- Tumhara Details ---\n");
    printf("Naam   : %s\n", name);
    printf("Age    : %d\n", age);
    printf("Marks  : %.2f\n", marks);     // 2 decimal tak
    printf("Grade  : %c\n", grade);
```

```
    return 0;
}
```

stdio.h vs conio.h (Quick Comparison)

Cheez	stdio.h	conio.h
Kaam	Standard Input/Output	Console control
printf/scanf	Haan	Nahi (alag functions)
getch()	Nahi	Haan
clrscr()	Nahi	Haan
Modern compilers	Perfect chalta hai	Nahi chalta (Turbo C only)
Use	Har program mein	Sirf old Turbo C mein

Aaj kal zyadatar log sirf stdio.h use karte hain.

✅ VARIABLE

Variable Kya Hai? (Sabse Simple Explanation)

Variable = **Memory ka chhota dabba** jismein hum value store karte hain.
Jaise ghar mein almirah hota hai na?

- Almirah ka naam = variable ka naam
- Almirah mein kya rakha hai = variable ki value
- Almirah kis size ka hai = variable ka data type

Example:

C

```
int age = 22;
```

- age → variable ka naam
- 22 → ismein stored value
- int → data type (pura number store kar sakta hai)

Variable Naming Rules (Yeh Rules Yaad Rakhna Zaroori Hai)

Rule	Sahi Example	Galat Example
Alphabet ya _ se shuru hona chahiye	age, _marks	1age, 2nd
Number beech mein allowed	marks1, rollNo	-
Space nahi allowed	mathsMarks	maths marks
Special characters nahi (sirf _)	total_amount	total@amount
Keyword nahi ho sakta	total	int, float

Best Practice: Variable ka naam meaningful hona chahiye jaise studentAge, totalMarks, firstName

C Language Complete Notes
Learn C programming from basics to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

✅ DATA TYPES IN C

Yeh C ke main data types hain:

Data Type	Size (approx)	Kya Store Kar Sakta Hai?	Example	Format Specifier
int	4 bytes	Pura number (-2 billion se +2 billion)	int age = 22;	%d
float	4 bytes	Decimal number (7 decimal digits)	float marks = 92.75;	%f
double	8 bytes	Lamba decimal (15 decimal digits)	double pi = 3.141592;	%lf
char	1 byte	Single character	char grade = 'A';	%c

Sabse Commonly Use Hone Wale:

- **int** → counting, age, marks (pura number)
- **float** → average, percentage, height
- **double** → jab bahut accuracy chahiye (jaise scientific calculation)
- **char** → single letter, symbol

Declaration aur Initialization

Declaration = Sirf dabba banana **Initialization** = Dabbe mein value daalna

C

```
int age;                // Declaration
age = 22;               // Assignment

int marks = 95;         // Declaration + Initialization (sabse best)
```

Multiple Variables Ek Line Mein:

C

```
int a, b, c;
float x = 3.14, y = 2.5;
```

Complete Practical Example

C

```
#include <stdio.h>

int main()
{
    int age = 22;
    float height = 5.8;
    char grade = 'A';
```

```c
    double salary = 45000.75;

    printf("Age = %d\n", age);
    printf("Height = %.2f feet\n", height);   // 2 decimal tak
    printf("Grade = %c\n", grade);
    printf("Salary = %.2lf\n", salary);

    return 0;
}
```

Learn C programming from basics to advanced with real-world examples

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

Output:

text

```
Age = 22
Height = 5.80 feet
Grade = A
Salary = 45000.75
```

Common Mistakes Beginners Karte Hain

1. Variable declare karte waqt data type bhool jaana
2. scanf mein & nahi lagana
3. char mein double quotes use karna ('A' hoga, "A" nahi)
4. Float mein %d use karna
5. Variable naam mein space daalna

✅ PRACTICE SET 1

Section 1: Variables & Data Types (10 Questions)

Q1. Ek program likho jismein int, float, char aur double type ke variables declare kiye jayein aur unki values print ki jayein.

C

```c
#include <stdio.h>
int main() {
    int age = 21;
    float height = 5.7;
    char grade = 'A';
    double salary = 45000.75;

    printf("Age = %d\n", age);
    printf("Height = %.2f\n", height);
    printf("Grade = %c\n", grade);
    printf("Salary = %.2lf\n", salary);
    return 0;
}
```

Output:

text

```
Age = 21
Height = 5.70
Grade = A
Salary = 45000.75
```

Q2. int x = 5.7; karne par kya hoga? Kyun?

Answer: x mein sirf 5 store hoga (decimal part cut ho jayega). Reason: int sirf pura number store kar sakta hai.

Q3. Ek program likho jismein user se int aur float input le aur dono ko print kare.

C

```c
#include <stdio.h>
int main() {
    int a;
    float b;
    printf("Enter integer: ");
    scanf("%d", &a);
    printf("Enter float: ");
    scanf("%f", &b);
    printf("Integer = %d, Float = %.2f", a, b);
    return 0;
}
```

Q4. char ch = 'A'; ko %d se print karne par kya aayega?

Answer: 65 (ASCII value of 'A')

Q5. float x = 3 / 2; karne par kya value aayegi?

Answer: 1.0 (kyunki dono taraf int hai, isliye integer division hoga)

Section 2: stdio.h (printf & scanf) (8 Questions)

Q6. Ek program likho jo user se naam, age aur marks le aur sab print kare.

C

```c
#include <stdio.h>
int main() {
    char name[50];
    int age;
    float marks;

    printf("Enter name: ");
    scanf("%s", name);
    printf("Enter age: ");
    scanf("%d", &age);
    printf("Enter marks: ");
    scanf("%f", &marks);

    printf("\nName: %s\nAge: %d\nMarks: %.2f", name, age, marks);
    return 0;
}
```

Q7. printf("%d %f %c", 10, 3.14, 'X'); ka output kya hoga?

Output: 10 3.140000 X

Q8. scanf mein & kyun lagana padta hai?

Answer: & variable ka address deta hai taaki scanf usmein value daal sake.

Q9. Ek program likho jo 3 integers le aur unka sum print kare.

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int a, b, c;
    printf("Enter 3 numbers: ");
    scanf("%d %d %d", &a, &b, &c);
    printf("Sum = %d", a + b + c);
    return 0;
}
```

Q10. printf("Hello\nWorld"); aur printf("Hello"); printf("\nWorld"); dono ka output same hai kya?

Answer: Haan, dono ka output same hai.

Learn C programming from basics to advanced

Section 3: conio.h (clrscr, getch, gotoxy etc.) (7 Questions)

Q11. clrscr() ka use kya hai?

Answer: Puri screen clear kar deta hai.

Q12. getch() aur getche() mein kya difference hai?

Answer:

- getch() → character leta hai lekin screen par nahi dikhata
- getche() → character leta hai aur screen par bhi dikhata hai

Q13. Ek program likho jo gotoxy(30, 10) use kare aur wahan "Welcome" print kare.

C

```c
#include <stdio.h>
#include <conio.h>
int main() {
    clrscr();
    gotoxy(30, 10);
    printf("Welcome");
    getch();
    return 0;
}
```

Q14. textcolor(RED); cprintf("Hello"); ka kya matlab hai?

Answer: Text ka color red kar deta hai aur "Hello" print karta hai.

Q15. kbhit() ka use kya hai?

Created by **Pawan Soni**

Answer: Check karta hai ki koi key press hui hai ya nahi (loop mein use hota hai).

Q16. delay(2000); ka matlab kya hai?

Answer: Program ko 2000 milliseconds (2 seconds) ke liye rok deta hai.

Q17. conio.h modern compilers (VS Code) mein kyun nahi chalta?

Answer: Yeh Turbo C ke liye bana tha. Modern GCC mein yeh header file nahi hoti.

Bonus 3 Questions (Extra Practice)

Q18. Ek program banao jo user se ek character le aur uski ASCII value print kare (conio.h + stdio.h dono use karke).

THE ARTIFICIAL EDUCATION

Q19. int x = 10; float y = x / 3; ka output kya hoga? Kyun?

Q20. printf("%d", 'A'); ka output kya hoga?

Answer: 65

✅ INSTRUCTIONS (STATEMENTS) AUR ✅ OPERATORS IN C

1. Instructions (Statements) Kya Hote Hain?

C language mein **Instruction** ya **Statement** wo line hoti hai jo computer ko koi kaam karne ka order deti hai.

Har statement ke end mein **semicolon (;)** lagana zaroori hai.

Main Types of Statements:

Statement Type	Kaam Kya Karta Hai	Example
Declaration Statement	Variable banana (dabba create karna)	int age;
Assignment Statement	Variable mein value daalna	age = 21;
Input Statement	User se value lena	scanf("%d", &age);
Output Statement	Screen pe value print karna	printf("Age = %d", age);
Control Statement	Decision lena (if-else, loops)	if(age >= 18)

Sabse Common Statements (Beginner ke liye):

C

```
int x;                  // Declaration
x = 10;                 // Assignment
scanf("%d", &x);        // Input
printf("%d", x);        // Output
```

2. Operators in C (Pura Detail)

Operator wo symbol hai jo variables ya values ke beech mein kaam karta hai (jaise + , - , * etc.).

C mein operators ke **5 main groups** hain:

A. Arithmetic Operators (Calculation ke liye)

Operator	Kaam	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	4 * 5	20
/	Division	10 / 2	5
%	Modulo (Remainder)	10 % 3	1

Important Note:

- 5 / 2 → agar dono int hain to answer 2 aayega (decimal ignore)
- 5.0 / 2 → answer 2.5 aayega

C Language Complete Notes
Learn C programming from basics to advanced
Created by Pawan Soni
Learn computer, control your future
THE ARTIFICIAL EDUCATION

B. Relational Operators (Comparison ke liye)

Operator	Matlab	Example
>	Greater than	5 > 3
<	Less than	3 < 5
>=	Greater than or equal	5 >= 5
<=	Less than or equal	4 <= 5
==	Equal to	5 == 5
!=	Not equal to	5 != 3

C. Logical Operators (AND, OR, NOT)

Operator	Symbol	Matlab	Example
AND	&&	Dono true	(5>3) && (10>2)
OR	`		`
NOT	!	Ulta kar do	!(5>3) → false

D. Assignment Operators (Value daalne ke liye)

Operator	Matlab	Example	Equivalent
=	Simple assignment	x = 10	-
+=	Add and assign	x += 5	x = x + 5
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 2	x = x * 2
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulo and assign	x %= 3	x = x % 3

E. Increment & Decrement Operators (Sabse Famous)

Operator	Kaam	Example
++	Increase by 1	x++
--	Decrease by 1	x--

Pre-increment vs Post-increment:

C

```
int x = 5;
printf("%d", ++x);    // Pehle badhao phir print → 6
printf("%d", x++);    // Pehle print phir badhao → 5 (phir x=6)
```

3. Operator Precedence (Hierarchy) – Sabse Important

C mein operators ki priority hoti hai (jaise BODMAS):

1. () → Brackets (sabse pehle)
2. ++ -- (Increment/Decrement)
3. * / % (Multiplication, Division, Modulo)
4. + - (Addition, Subtraction)

C Language Complete Notes
Learn C programming from basics to advanced with real world examples
THE ARTIFICIAL EDUCATION

5. Relational (>, <, == etc.)
6. Logical (&&, ||)
7. Assignment (=, += etc.)

Example:

C

```
int result = 10 + 20 * 2;      // Pehle 20*2 = 40, phir 10+40 = 50
```

✅ PRACTICE SET 2

1. Basic Declaration + Assignment Statement

C

```
#include <stdio.h>
int main() {
    int age;            // Declaration
    age = 21;           // Assignment
    printf("Age = %d", age);
    return 0;
}
```

Output: Age = 21 **Explanation:** Pehle variable banaya, phir usme value daali.

2. Declaration + Initialization (Ek Line Mein)

C

```
#include <stdio.h>
int main() {
    int marks = 95;           // Declaration + Initialization
    float height = 5.8;
    printf("Marks = %d\nHeight = %.2f", marks, height);
    return 0;
}
```

Output:

```text
Marks = 95
Height = 5.80
```

3. Input + Output using scanf & printf

C

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Do numbers daalo: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d", a + b);
```

```
    return 0;
}
```

Sample Input: 10 20 **Output:** Sum = 30

***4. Arithmetic Operators (+, -, , /, %)**

C

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Add = %d\n", a + b);
    printf("Sub = %d\n", a - b);
    printf("Mul = %d\n", a * b);
    printf("Div = %d\n", a / b);      // 3 (decimal ignore)
    printf("Mod = %d\n", a % b);      // 1 (remainder)
    return 0;
}
```

Output:

text

```
Add = 13
Sub = 7
Mul = 30
Div = 3
Mod = 1
```

5. Integer vs Float Division

C

```
#include <stdio.h>
int main() {
    printf("5 / 2 = %d\n", 5 / 2);          // int division
    printf("5.0 / 2 = %.2f\n", 5.0 / 2);    // float division
    return 0;
}
```

Output:

text

```
5 / 2 = 2
5.0 / 2 = 2.50
```

6. Relational Operators (Comparison)

C

```
#include <stdio.h>
int main() {
    int a = 10, b = 5;
```

```
    printf("a > b  = %d\n", a > b);      // 1 (true)
    printf("a < b  = %d\n", a < b);      // 0 (false)
    printf("a == b = %d\n", a == b);     // 0
    printf("a != b = %d\n", a != b);     // 1
    return 0;
}
```

Output:

text

```
a > b  = 1
a < b  = 0
a == b = 0
a != b = 1
```

7. Logical Operators (AND, OR, NOT)

C

```
#include <stdio.h>
int main() {
    int age = 20;
    printf("Adult? = %d\n", (age >= 18) && (age <= 60));   // AND
    printf("Teen?  = %d\n", (age < 13) || (age > 19));     // OR
    printf("Not adult? = %d\n", !(age >= 18));             // NOT
    return 0;
}
```

Output:

text

```
Adult? = 1
Teen?  = 0
Not adult? = 0
```

8. Compound Assignment Operators

C

```
#include <stdio.h>
int main() {
    int x = 10;
    x += 5;     // x = x + 5
    x *= 2;     // x = x * 2
    x %= 7;     // x = x % 7
    printf("Final x = %d", x);
    return 0;
}
```

Output: Final x = 4

Complete Notes
THE ARTIFICIAL EDUCATION

9. Increment & Decrement Operators

C

```c
#include <stdio.h>
int main() {
    int x = 5;
    printf("x++ = %d\n", x++);   // Post-increment
    printf("++x = %d\n", ++x);   // Pre-increment
    printf("--x = %d\n", --x);
    return 0;
}
```

Output:

text

```
x++ = 5
++x = 7
--x = 6
```

10. Operator Precedence (BODMAS in C)

C

```c
#include <stdio.h>
int main() {
    int result = 10 + 20 * 2 / 4 - 3 % 2;
    printf("Result = %d", result);
    return 0;
}
```

Output: Result = 17 **Explanation:** *, /, % pehle hote hain, phir + aur -.

11. Mixed Operators (Real Life Example)

C

```c
#include <stdio.h>
int main() {
    int principal = 1000;
    float rate = 5.5;
    int time = 2;
    float si = (principal * rate * time) / 100;
    printf("Simple Interest = %.2f", si);
    return 0;
}
```

Output: Simple Interest = 110.00

12. Bonus: User se input leke operators use karna

C

Learn C programming from basics to advanced with real-world examples

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int a, b;
    printf("Do numbers daalo: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d\n", a + b);
    printf("Product = %d\n", a * b);
    printf("Remainder = %d\n", a % b);
    return 0;
}
```

✅ CONDITIONAL STATEMENTS

Conditional Statements wo cheezein hain jo program ko **decision** lene detin hain. Matlab: "Agar yeh condition sahi hai toh yeh karo, warna woh karo."

C mein sabse zyada use hone wale conditional statements yeh hain:

1. if
2. if-else
3. else-if ladder
4. switch-case

Ab ek-ek ko **dheere-dheere, practical examples ke saath** samjhaata hoon.

1. if Statement (Sabse Basic)

Syntax:

C

```c
if (condition) {
    // agar condition sach hai toh yeh code chalega
}
```

Example 1:

C

```c
#include <stdio.h>
int main() {
    int age = 20;
    if (age >= 18) {
        printf("Tum adult ho!\n");
    }
    return 0;
}
```

Output: Tum adult ho!

2. if-else Statement

Agar condition sach hai → if block Galat hai → else block

Example 2:

C

```c
#include <stdio.h>
int main() {
    int age;
    printf("Apni age batao: ");
    scanf("%d", &age);

    if (age >= 18) {
        printf("Adult ho!\n");
    } else {
        printf("Abhi bacche ho!\n");
    }
    return 0;
}
```

3. else-if Ladder (Multiple Conditions)

Jab 2 se zyada conditions ho.

Example 3:

C

```c
#include <stdio.h>
int main() {
    int marks;
    printf("Apne marks batao: ");
    scanf("%d", &marks);

    if (marks >= 90) {
        printf("Grade: A+\n");
    } else if (marks >= 80) {
        printf("Grade: A\n");
    } else if (marks >= 70) {
        printf("Grade: B\n");
    } else {
        printf("Grade: C ya Fail\n");
    }
    return 0;
}
```

4. switch-case Statement

Ek variable ke alag-alag values ke hisaab se kaam karne ke liye.

Example 4:

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int day;
    printf("Enter day number (1-7): ");
    scanf("%d", &day);

    switch(day) {
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
        case 5: printf("Friday\n"); break;
        case 6: printf("Saturday\n"); break;
        case 7: printf("Sunday\n"); break;
        default: printf("Invalid day!\n");
    }
    return 0;
}
```

Note: Har case ke baad break; lagana bahut zaroori hai.

✅ PRACTICE SET 3

Q1. User se number lo aur batao woh even hai ya odd.

C

```c
#include <stdio.h>
int main() {
    int n;
    printf("Number daalo: ");
    scanf("%d", &n);
    if(n % 2 == 0)
        printf("Even hai");
    else
        printf("Odd hai");
    return 0;
}
```

Q2. Teen numbers mein se sabse bada print karo.

C

```c
#include <stdio.h>
int main() {
    int a, b, c;
    printf("3 numbers daalo: ");
    scanf("%d %d %d", &a, &b, &c);

    if(a > b && a > c)
```

```
        printf("%d sabse bada hai", a);
    else if(b > c)
        printf("%d sabse bada hai", b);
    else
        printf("%d sabse bada hai", c);
    return 0;
}
```

Q3. User se character lo aur batao woh vowel hai ya consonant.

C

```
#include <stdio.h>
int main() {
    char ch;
    printf("Ek character daalo: ");
    scanf(" %c", &ch);

    if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u' ||
       ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U')
        printf("Vowel hai");
    else
        printf("Consonant hai");
    return 0;
}
```

Q4. Simple calculator (switch-case use karke)

C

```
#include <stdio.h>
int main() {
    char op;
    int a, b;
    printf("Do numbers aur operator daalo (+ - * /): ");
    scanf("%d %c %d", &a, &op, &b);

    switch(op) {
        case '+': printf("%d", a+b); break;
        case '-': printf("%d", a-b); break;
        case '*': printf("%d", a*b); break;
        case '/': printf("%d", a/b); break;
        default: printf("Galat operator");
    }
    return 0;
}
```

Q5. Check whether a number is Positive, Negative or Zero

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int num;
    printf("Koi number daalo: ");
    scanf("%d", &num);

    if(num > 0)
        printf("Positive hai");
    else if(num < 0)
        printf("Negative hai");
    else
        printf("Zero hai");
    return 0;
}
```

Sample Input: 5 → **Output:** Positive hai **Sample Input:** -3 → **Output:** Negative hai

Q6. Check if a year is Leap Year or not

C

```c
#include <stdio.h>
int main() {
    int year;
    printf("Year daalo: ");
    scanf("%d", &year);

    if((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d Leap Year hai", year);
    else
        printf("%d Normal Year hai", year);
    return 0;
}
```

Sample Input: 2024 → **Output:** 2024 Leap Year hai

Q7. Student Grade System (else-if ladder)

C

```c
#include <stdio.h>
int main() {
    int marks;
    printf("Marks daalo (0-100): ");
    scanf("%d", &marks);

    if(marks >= 90)
        printf("Grade: A+");
```

Created by Pawan Soni
learn computer control your future
THE ARTIFICIAL EDUCATION

```
    else if(marks >= 80)
        printf("Grade: A");
    else if(marks >= 70)
        printf("Grade: B");
    else if(marks >= 60)
        printf("Grade: C");
    else
        printf("Grade: Fail");
    return 0;
}
```

Learn C programming from basics to advanced
with real-world examples

Q8. Check type of Triangle from 3 sides

C

```
#include <stdio.h>
int main() {
    int a, b, c;
    printf("3 sides daalo: ");
    scanf("%d %d %d", &a, &b, &c);

    if(a == b && b == c)
        printf("Equilateral Triangle");
    else if(a == b || b == c || a == c)
        printf("Isosceles Triangle");
    else
        printf("Scalene Triangle");
    return 0;
}
```

Q9. Simple Calculator using switch-case

C

```
#include <stdio.h>
int main() {
    char op;
    int a, b;
    printf("Do numbers aur operator daalo (+ - * /): ");
    scanf("%d %c %d", &a, &op, &b);

    switch(op) {
        case '+': printf("Result = %d", a+b); break;
        case '-': printf("Result = %d", a-b); break;
        case '*': printf("Result = %d", a*b); break;
        case '/': printf("Result = %d", a/b); break;
```

THE ARTIFICIAL EDUCATION

```
        default: printf("Galat operator!");
    }
    return 0;
}
```

Q10. Vowel or Consonant using switch-case

C

```
#include <stdio.h>
int main() {
    char ch;
    printf("Ek character daalo: ");
    scanf(" %c", &ch);

    switch(ch) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            printf("Vowel hai");
            break;
        default:
            printf("Consonant hai");
    }
    return 0;
}
```

Extra Bonus Questions (Q11 & Q12)

Q11. Nested if – Check eligibility for voting + driving

C

```
#include <stdio.h>
int main() {
    int age;
    printf("Age daalo: ");
    scanf("%d", &age);

    if(age >= 18) {
        printf("Voting ke liye eligible ho\n");
        if(age >= 21)
            printf("Driving license bhi le sakte ho");
    } else {
        printf("Voting ke liye abhi eligible nahi ho");
    }
    return 0;
}
```

Q12. Menu based program using switch-case (ATM style)

C

```
#include <stdio.h>
int main() {
    int choice;
    printf("1. Balance Check\n2. Withdraw\n3. Deposit\nChoice daalo: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1: printf("Aapka balance: Rs 5000"); break;
        case 2: printf("Withdraw successful"); break;
        case 3: printf("Deposit successful"); break;
        default: printf("Invalid choice");
    }
    return 0;
}
```

✅ LOOP CONTROL STATEMENTS

Loop Control Statements wo hain jo program mein **baar-baar** (repeat) kaam karne ke liye use hote hain. Jaise school mein "1 se 100 tak numbers likho" – har baar manually nahi likh sakte, isliye loop use karte hain.

C mein **3 main loops** hote hain:

1. for loop
2. while loop
3. do-while loop

Aur inke control ke liye 2 important keywords:

- break
- continue

Ab ek-ek ko **dheere-dheere** samjhaata hoon.

1. for Loop (Sabse Zyada Use Hota Hai)

Syntax:

C

```
for(initialization; condition; updation) {
    // jo baar-baar karna hai
}
```

Practical Example 1: 1 se 10 tak numbers print karo

C

```
#include <stdio.h>
int main() {
    int i;
```

Created by Pawan Soni
learn computer, control your future

```
    for(i = 1; i <= 10; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Output:

text

```
1
2
3
...
10
```

Explanation:

- i = 1 → initialization (shuruat)
- i <= 10 → condition (jab tak yeh sach hai, loop chalega)
- i++ → updation (har baar i ko +1 kar do)

2. while Loop

Syntax:

C

```
while(condition) {
    // code
}
```

Example 2: 1 se 5 tak print karo (while se)

C

```
#include <stdio.h>
int main() {
    int i = 1;
    while(i <= 5) {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

Difference from for loop: for mein initialization, condition aur updation ek hi line mein likhe jaate hain. while mein alag-alag.

3. do-while Loop (Kam se kam ek baar toh chalega)

Syntax:

C

THE ARTIFICIAL EDUCATION

```
do {
    // code
} while(condition);
```

Example 3:

C

```
#include <stdio.h>
int main() {
    int i = 1;
    do {
        printf("%d\n", i);
        i++;
    } while(i <= 5);
    return 0;
}
```

Important Point: do-while mein condition last mein check hoti hai, isliye loop **kam se kam ek baar** zaroor chalega.

4. Loop Control Keywords

break → Loop ko turant rok deta hai

Example:

C

```
for(int i = 1; i <= 10; i++) {
    if(i == 6)
        break;          // 6 pe loop ruk jayega
    printf("%d\n", i);
}
```

Output: 1 2 3 4 5

continue → Current iteration skip karke agla round chala deta hai

Example:

C

```
for(int i = 1; i <= 8; i++) {
    if(i == 4 || i == 6)
        continue;       // 4 aur 6 ko skip
    printf("%d\n", i);
}
```

Output: 1 2 3 5 7 8

✅ PRACTICE SET 4

Q1. 1 se 50 tak sabhi even numbers print karo (for loop use karke)

Answer:

C

THE ARTIFICIAL EDUCATION

```
for(int i = 2; i <= 50; i += 2) {
    printf("%d\n", i);
}
```

Q2. User se number lo aur uska table (multiplication table) print karo.

C

```
#include <stdio.h>
int main() {
    int n;
    printf("Number daalo: ");
    scanf("%d", &n);
    for(int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n*i);
    }
    return 0;
}
```

Q3. 1 se 100 tak numbers ka sum calculate karo (while loop se)

C

```
int sum = 0, i = 1;
while(i <= 100) {
    sum += i;
    i++;
}
printf("Sum = %d", sum);   // Output: 5050
```

Q4. User se numbers lete raho jab tak user -1 nahi daalta. Sab numbers ka sum print karo.

C

```
#include <stdio.h>
int main() {
    int num, sum = 0;
    printf("Numbers daalo (-1 dabakar stop karo):\n");
    while(1) {
        scanf("%d", &num);
        if(num == -1) break;
        sum += num;
    }
    printf("Total Sum = %d", sum);
    return 0;
}
```

Q5. 1 se 20 tak numbers print karo lekin 13 skip karo (continue use karke)

Q6. Nested loop: 5x5 star pattern print karo

C

```
for(int i = 1; i <= 5; i++) {
    for(int j = 1; j <= 5; j++) {
```

```
        printf("* ");
    }
    printf("\n");
}
```

Q7. Factorial of a number (user input)
Q8. Prime number check karo (1 se n tak loop chalao)
Q9. Reverse of a number (while loop se)
Q10. Menu driven program using do-while + switch (Calculator jaisa)

✅ FUNCTIONS

1. Functions Kya Hote Hain?

Function = **Chhota program** jo ek specific kaam karta hai aur baar-baar use kiya ja sakta hai.
Jaise ghar mein alag-alag kaam ke liye alag-alag log hote hain (rasoi, safai, bill payment), waise hi C mein alag-alag functions hote hain.

Fayde:

- Code repeat nahi hota
- Code clean aur organized rehta hai
- Debugging easy hoti hai
- Team mein kaam karna aasan hota hai

2. Function Ka Structure

Ek function 3 parts mein bana hota hai:

1. **Declaration** (Sirf bata dena ki function hai)
2. **Definition** (Asli code likhna)
3. **Call** (Function ko use karna)

Syntax (User-defined Function):

C

```c
// Declaration (prototype)
return_type function_name (data_type param1, data_type param2, ...);

// Definition
return_type function_name (data_type param1, data_type param2, ...) {
    // function ka code
    return value;   // optional
}
```

3. Practical Examples of Functions

Example 1: Simple Function (No argument, No return)

C

```c
#include <stdio.h>
```

```c
void printHello() {          // Definition
    printf("Hello World!\n");
}

int main() {
    printHello();            // Call
    printHello();            // Call again
    return 0;
}
```

Output:

```text
Hello World!
Hello World!
```

Example 2: Function with Argument (Parameter)

C

```c
#include <stdio.h>

void printTable(int n) {     // n is parameter
    for(int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n*i);
    }
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);
    printTable(num);         // num is argument
    return 0;
}
```

Example 3: Function with Return Value

C

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int sum = add(10, 25);   // function call
    printf("Sum = %d", sum);
```

```
    return 0;
}
```

Output: Sum = 35

C Language

Complete Notes

Learn C programming from basics to advanced with real-world examples

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

4. Types of Functions

Type	Example
No argument, No return	void printHello()
With argument, No return	void printTable(int n)
With argument, With return	int add(int a, int b)
No argument, With return	int getNumber()

5. Recursion Kya Hai?

Recursion = Jab ek function **khud ko call** karta hai.

Important Rule:

- Har recursive function mein **Base Case** hona chahiye (jahan recursion ruk jaaye)
- Warna infinite loop ban jaayega (stack overflow error)

6. Practical Examples of Recursion

Example 1: Factorial using Recursion

C

```
#include <stdio.h>

int factorial(int n) {
    if(n == 0 || n == 1)          // Base Case
        return 1;
    else
        return n * factorial(n-1); // Recursive Call
}

int main() {
    int num = 5;
    printf("Factorial of %d = %d", num, factorial(num));
    return 0;
}
```

Output: Factorial of 5 = 120

Example 2: Sum of First N Natural Numbers

C

```
int sum(int n) {
    if(n == 0)
        return 0;
    return n + sum(n-1);
}
```

Example 3: Fibonacci Series (Classic Recursion)

C

```
int fib(int n) {
    if(n <= 1)
        return n;
    return fib(n-1) + fib(n-2);
}
```

✅ PRACTICE SET 5

Q1. Ek function banao jo do numbers ka sum return kare.

C

```
#include <stdio.h>

int sum(int a, int b) {
    return a + b;
}

int main() {
    int x, y;
    printf("Do numbers daalo: ");
    scanf("%d %d", &x, &y);
    printf("Sum = %d", sum(x, y));
    return 0;
}
```

Sample Input: 10 25 **Output:** Sum = 35

Q2. Ek function banao jo number ka square return kare.

C

```
#include <stdio.h>

int square(int n) {
    return n * n;
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);
    printf("Square = %d", square(num));
    return 0;
}
```

Sample Input: 7 **Output:** Square = 49

Complete Notes
THE ARTIFICIAL EDUCATION

Q3. Ek function banao jo user se n le aur uska table print kare.

C

```c
#include <stdio.h>

void printTable(int n) {
    for(int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);
    printTable(num);
    return 0;
}
```

Q4. Recursion se 1 se n tak ka sum nikaalo.

C

```c
#include <stdio.h>

int sum(int n) {
    if(n == 0)
        return 0;
    return n + sum(n - 1);
}

int main() {
    int n;
    printf("N daalo: ");
    scanf("%d", &n);
    printf("Sum = %d", sum(n));
    return 0;
}
```

Sample Input: 5 **Output:** Sum = 15

Q5. Recursion se number ka factorial calculate karo.

C

```c
#include <stdio.h>

int factorial(int n) {
```

```
    if(n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    int n;
    printf("Number daalo: ");
    scanf("%d", &n);
    printf("Factorial = %d", factorial(n));
    return 0;
}
```

Sample Input: 5 **Output:** Factorial = 120

Q6. Ek function banao jo check kare ki number prime hai ya nahi.

C

```
#include <stdio.h>

int isPrime(int n) {
    if(n <= 1) return 0;
    for(int i = 2; i * i <= n; i++) {
        if(n % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);
    if(isPrime(num))
        printf("%d Prime hai", num);
    else
        printf("%d Prime nahi hai", num);
    return 0;
}
```

Q7. Recursion se Fibonacci series ke n terms print karo.

C

```
#include <stdio.h>
```

```c
int fib(int n) {
    if(n <= 1)
        return n;
    return fib(n-1) + fib(n-2);
}

int main() {
    int n;
    printf("Kitne terms chahiye: ");
    scanf("%d", &n);
    printf("Fibonacci Series: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", fib(i));
    }
    return 0;
}
```

Sample Input: 8 **Output:** 0 1 1 2 3 5 8 13

Q8. Ek function banao jo string ki length return kare (without using built-in function).

C

```c
#include <stdio.h>

int strLength(char str[]) {
    int len = 0;
    while(str[len] != '\0') {
        len++;
    }
    return len;
}

int main() {
    char name[50];
    printf("String daalo: ");
    scanf("%s", name);
    printf("Length = %d", strLength(name));
    return 0;
}
```

Q9. Recursion se number ko reverse karo.

C

```c
#include <stdio.h>
```

```c
void reverse(int n) {
    if(n == 0) return;
    printf("%d", n % 10);
    reverse(n / 10);
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);
    printf("Reverse = ");
    reverse(num);
    return 0;
}
```

Sample Input: 12345 **Output:** Reverse = 54321

Q10. Ek menu-driven program banao jismein different functions call ho (add, subtract, multiply, divide).

C

```c
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }
int divide(int a, int b) { return (b != 0) ? a / b : 0; }

int main() {
    int choice, x, y;
    while(1) {
        printf("\n1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Exit\nChoice:
");
        scanf("%d", &choice);

        if(choice == 5) break;

        printf("Do numbers daalo: ");
        scanf("%d %d", &x, &y);

        switch(choice) {
            case 1: printf("Result = %d", add(x, y)); break;
            case 2: printf("Result = %d", sub(x, y)); break;
            case 3: printf("Result = %d", mul(x, y)); break;
```

```
            case 4: printf("Result = %d", divide(x, y)); break;
            default: printf("Galat choice!");
        }
    }
    return 0;
}
```

✅ POINTERS

Pointers C language ka sabse important aur powerful concept hai. Yeh thoda confusing lagta hai shuru mein, lekin ek baar samajh aa gaya toh baaki sab cheezein (arrays, functions, strings, dynamic memory) bahut easy ho jaati hain.

1. Pointer Kya Hai? (Sabse Simple Explanation)

Pointer ek variable hai jo **kisi aur variable ka address** (ghar ka pata) store karta hai.

- Normal variable → value store karta hai (jaise 25)
- Pointer → us value ka **address** store karta hai (jaise 0x7ffd1234)

Analogy:

- Ghar mein ek dabba hai jiska naam age hai aur usme 21 likha hai.
- Pointer = dabbe ka **address** (ghar ka number plate) hold karta hai.

2. Pointer Kyun Zaroori Hai?

- Memory ko directly control kar sakte ho
- Arrays ke saath bahut powerful hota hai
- Functions mein call by reference (value change kar sakte ho)
- Dynamic memory allocation (malloc) ke liye
- Strings aur data structures mein use hota hai

3. Pointer Ka Syntax (Declaration)

C

```
data_type *pointer_name;
```

Example:

C

```
int *ptr;        // ptr ek pointer hai jo int type ke variable ka address store karega
float *fptr;     // float ke liye
char *cptr;      // char ke liye
```

Initialization:

C

Complete Notes
Learn C programming from basic to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
int x = 10;
int *ptr = &x;      // & = address of operator
```

4. Important Operators

Operator	Naam	Kaam
&	Address of Operator	Variable ka address deta hai
*	Dereference Operator	Address se value nikaalta hai

Example:

C

```
#include <stdio.h>
int main() {
    int x = 25;
    int *ptr = &x;

    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", &x);      // %p = address print karne ke liye
    printf("Pointer ptr = %p\n", ptr);
    printf("Value through pointer = %d\n", *ptr);   // *ptr = value nikaalo

    return 0;
}
```

5. Pointers ke Practical Examples

Example 1: Simple Pointer

C

```
#include <stdio.h>
int main() {
    int a = 100;
    int *p = &a;

    printf("a = %d\n", a);
    printf("*p = %d\n", *p);

    *p = 200;        // value change kar di through pointer
    printf("Ab a = %d\n", a);
    return 0;
}
```

Example 2: Pointer Arithmetic

C

C Language Complete Notes
Learn C programming from basics to advanced
THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    printf("%d\n", *ptr);       // 10
    ptr++;
    printf("%d\n", *ptr);       // 20
    ptr++;
    printf("%d\n", *ptr);       // 30
    return 0;
}
```

6. Pointers and Arrays (Sabse Important)

Array ka naam khud hi uska first element ka address hota hai.

C

```c
int arr[5] = {1, 2, 3, 4, 5};
int *p = arr;     // p = &arr[0]

printf("%d", *(p + 2));   // 3
```

7. Pointers and Functions (Call by Reference)

C

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 10, b = 20;
    swap(&a, &b);
    printf("a = %d, b = %d", a, b);   // 20 10
    return 0;
}
```

8. NULL Pointer

C

```c
int *ptr = NULL;     // safe pointer
```

THE ARTIFICIAL EDUCATION

✅ PRACTICE SET 6

Q1. Ek program likho jo ek integer variable ka address print kare.

C

```c
#include <stdio.h>
int main() {
    int num = 100;
    printf("Value of num = %d\n", num);
    printf("Address of num = %p\n", &num);   // %p se address print hota hai
    return 0;
}
```

Output:

text

```text
Value of num = 100
Address of num = 0x7ffd1234abcd   (address har baar alag ho sakta hai)
```

Explanation: &num se variable ka memory address milta hai.

Q2. Do integers ke values ko pointer ke through swap karo (call by reference).

C

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Pehle: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("Baad mein: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output:

text

```text
Pehle: x = 10, y = 20
Baad mein: x = 20, y = 10
```

Explanation: Pointer ke through original value change ho jaati hai (call by reference).

Q3. Ek array ke sabhi elements ko pointer ke through print karo.

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;          // ptr points to first element

    for(int i = 0; i < 5; i++) {
        printf("%d ", *ptr);
        ptr++;               // pointer ko agle element par le jaate hain
    }
    return 0;
}
```

Output: 10 20 30 40 50

with real-world examples

Q4. Pointer arithmetic use karke array ke 3rd element ko print karo.

C

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    printf("3rd element = %d\n", *(ptr + 2));   // 0-based indexing
    return 0;
}
```

Output: 3rd element = 30

Q5. Ek function banao jo pointer ke through number ko double kar de.

C

```c
#include <stdio.h>

void doubleValue(int *num) {
    *num = *num * 2;
}

int main() {
    int x = 7;
    printf("Pehle: %d\n", x);
    doubleValue(&x);
    printf("Baad mein: %d\n", x);
    return 0;
}
```

Output:

text

THE ARTIFICIAL EDUCATION

```
Pehle: 7
Baad mein: 14
```

Q6. User se string le aur pointer ke through uski length calculate karo.

C

```c
#include <stdio.h>

int strLength(char *str) {
    int len = 0;
    while(*str != '\0') {
        len++;
        str++;
    }
    return len;
}

int main() {
    char name[50];
    printf("Apna naam daalo: ");
    scanf("%s", name);
    printf("Length = %d", strLength(name));
    return 0;
}
```

Q7. Ek integer pointer ko NULL initialize karke check karo.

C

```c
#include <stdio.h>
int main() {
    int *ptr = NULL;

    if(ptr == NULL)
        printf("Pointer NULL hai (safe)\n");
    else
        printf("Pointer NULL nahi hai\n");

    return 0;
}
```

Output: Pointer NULL hai (safe)

Q8. Recursion ke saath pointer use karke factorial calculate karo (advanced).

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>

void factorial(int n, int *result) {
    if(n == 0 || n == 1) {
        *result = 1;
        return;
    }
    factorial(n-1, result);
    *result = *result * n;
}

int main() {
    int num, fact = 1;
    printf("Number daalo: ");
    scanf("%d", &num);
    factorial(num, &fact);
    printf("Factorial = %d", fact);
    return 0;
}
```

Q9. Ek array ka maximum element pointer ke through nikaalo.

C

```c
#include <stdio.h>
int main() {
    int arr[5] = {12, 45, 7, 89, 23};
    int *ptr = arr;
    int max = *ptr;

    for(int i = 1; i < 5; i++) {
        if(*(ptr + i) > max)
            max = *(ptr + i);
    }
    printf("Maximum element = %d", max);
    return 0;
}
```

Output: Maximum element = 89

Q10. Do pointers ko add karke unka sum print karo.

C

```c
#include <stdio.h>
int main() {
    int a = 10, b = 20;
```

```
    int *p1 = &a;
    int *p2 = &b;

    int sum = *p1 + *p2;      // values ko add kar rahe hain
    printf("Sum of values = %d", sum);
    return 0;
}
```

Output: Sum of values = 30

✅ ARRAYS IN C

Bhai, yeh **Arrays** ka **poora aur detailed** tutorial hai. Main ne kuch bhi chhoda nahi hai. Har cheez step-by-step, simple Hinglish mein, examples ke saath di hai.

1. Array Kya Hai?

Array ek **same data type** ke multiple values ko **ek saath** store karne ka tarika hai.

- Normal variable → sirf **ek** value store kar sakta hai.
- Array → **kai** values (ek hi naam ke neeche) store kar sakta hai.

Example:

- int age = 21; → sirf ek age
- int ages[5] = {18, 21, 19, 22, 20}; → 5 alag-alag ages ek saath

2. Array Kyun Use Karte Hain?

- Code short aur clean rehta hai
- Same type ke data ko ek saath manage karna aasan
- Loops ke saath bahut powerful
- Functions mein pass karna aasan
- Pointers aur memory management ke liye base

3. 1D Array (One-Dimensional Array)

Declaration

C

```
data_type array_name[size];
```

Examples:

C

```
int marks[5];           // 5 integers store kar sakta hai
float salary[10];
char name[50];          // string ke liye bhi use hota hai
```

Initialization

C

Complete Notes
Learn C Programming from basics to advanced
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
int marks[5] = {85, 92, 78, 95, 88};      // size ke saath
int arr[] = {10, 20, 30};                  // size automatically decide hoti hai
int arr[5] = {0};                          // sab 0 ho jayenge
```

4. Array Elements Access Karna

Array 0-based indexing use karta hai (pehle element ka index 0 hota hai).

C

```
int arr[5] = {10, 20, 30, 40, 50};

printf("%d", arr[0]);   // 10
printf("%d", arr[3]);   // 40
printf("%d", arr[4]);   // 50
```

Change value:

C

```
arr[2] = 99;   // 3rd element ab 99 ho gaya
```

5. Memory Representation

Array ke elements **continuous (ek ke baad ek)** memory mein store hote hain.

- int arr[5]; → 5 integers → 5 × 4 = 20 bytes (generally)
- arr[0] ka address sabse pehle hota hai
- arr[1] = arr[0] + 4 bytes (int ke case mein)

6. Loops ke Saath Array

C

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // Print all elements
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output: 10 20 30 40 50

7. Passing Array to Function

Array ko function mein **call by reference** ki tarah pass kiya jaata hai.

C

```
#include <stdio.h>
```

```
void printArray(int arr[], int size) {     // arr[] likh sakte hain
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

int main() {
    int marks[4] = {85, 90, 78, 95};
    printArray(marks, 4);
    return 0;
}
```

8. Pointers aur Arrays ka Relation (Bahut Important)

Array ka naam khud hi uska **first element ka address** hota hai.

C

```
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr;     // ptr = &arr[0]

printf("%d", *(ptr + 2));   // 30
```

Note: arr[i] aur *(arr + i) dono same hain.

9. 2D Arrays (Two-Dimensional Arrays)

2D array = Matrix (rows × columns)

Declaration:

C

```
int matrix[3][4];     // 3 rows, 4 columns
```

Initialization:

C

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Access:

C

```
matrix[0][0] = 10;   // first row, first column
```

Print 2D Array:

C

```
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
```

learn computer, control your future
THE ARTIFICIAL EDUCATION

```
    printf("\n");
}
```

✅ PRACTICE SET 7

Q1. Ek array declare karo, user se 5 numbers input lo aur unhe print karo.

C

```
#include <stdio.h>
int main() {
    int arr[5];
    printf("5 numbers daalo:\n");
    for(int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Array elements: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Sample Input: 10 20 30 40 50 **Output:** Array elements: 10 20 30 40 50

Q2. Array ke sabhi elements ka sum calculate karo.

C

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int sum = 0;
    for(int i = 0; i < 5; i++) {
        sum += arr[i];
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Output: Sum = 150

Q3. Array mein sabse bada aur sabse chhota number nikaalo.

C

```
#include <stdio.h>
int main() {
    int arr[5] = {45, 12, 89, 33, 67};
    int max = arr[0], min = arr[0];
    for(int i = 1; i < 5; i++) {
```

```
        if(arr[i] > max) max = arr[i];
        if(arr[i] < min) min = arr[i];
    }
    printf("Maximum = %d\nMinimum = %d", max, min);
    return 0;
}
```

Output:

text

```
Maximum = 89
Minimum = 12
```

Q4. Array ko reverse karo (without using extra array).

C

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    for(int i = 0; i < 5/2; i++) {
        int temp = arr[i];
        arr[i] = arr[4 - i];
        arr[4 - i] = temp;
    }
    printf("Reversed Array: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output: Reversed Array: 50 40 30 20 10

Q5. Array mein even aur odd numbers ko count karo.

C

```
#include <stdio.h>
int main() {
    int arr[6] = {12, 7, 45, 22, 19, 8};
    int even = 0, odd = 0;
    for(int i = 0; i < 6; i++) {
        if(arr[i] % 2 == 0) even++;
        else odd++;
    }
    printf("Even = %d, Odd = %d", even, odd);
    return 0;
}
```

Output: Even = 3, Odd = 3

Q6. Linear Search – Array mein number dhundo aur uska index batao.

C

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 25, 30, 45, 60};
    int key, found = -1;
    printf("Search karne wala number daalo: ");
    scanf("%d", &key);
    for(int i = 0; i < 5; i++) {
        if(arr[i] == key) {
            found = i;
            break;
        }
    }
    if(found != -1)
        printf("Number %d index %d par mila", key, found);
    else
        printf("Number nahi mila");
    return 0;
}
```

Q7. 2D Array (Matrix) input lo aur print karo.

C

```c
#include <stdio.h>
int main() {
    int mat[2][3];
    printf("2x3 matrix ke elements daalo:\n");
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            scanf("%d", &mat[i][j]);
        }
    }
    printf("Matrix:\n");
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Q8. Do matrices ka addition karo (2D Array).

C

```c
#include <stdio.h>
int main() {
    int a[2][2] = {{1,2},{3,4}};
    int b[2][2] = {{5,6},{7,8}};
    int sum[2][2];

    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            sum[i][j] = a[i][j] + b[i][j];
        }
    }
    printf("Sum Matrix:\n");
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Q9. Array ko function mein pass karke uska sum return karo.

C

```c
#include <stdio.h>

int arraySum(int arr[], int size) {
    int sum = 0;
    for(int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    printf("Sum = %d", arraySum(arr, 5));
    return 0;
}
```

Output: Sum = 150

THE ARTIFICIAL EDUCATION

Q10. Pointer ke through array ke sabhi elements print karo.

C

```
#include <stdio.h>
int main() {
    int arr[5] = {100, 200, 300, 400, 500};
    int *ptr = arr;
    for(int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i));
    }
    return 0;
}
```

Output: 100 200 300 400 500

✅ STRINGS IN C

1. String Kya Hai?

String = **Characters ka group** (ek se zyada characters) jo ek saath store kiye jaate hain.

C mein string **character array** hoti hai aur **last mein null character '\0'** (null terminator) hona zaroori hai.

Example:

- "Pawan" → P a w a n \0
- "Hello" → H e l l o \0

Important Point: '\0' string ko end batata hai. Agar yeh nahi hoga toh program galat behave karega.

2. String Declaration & Initialization

Method 1: Size ke saath

C

```
char str[20];                    // 20 characters tak store kar sakta hai
char name[10] = "Rahul";         // automatically \0 add ho jaata hai
```

Method 2: Size ke bina

C

```
char name[] = "Pawan";           // Size automatically 6 ho jayegi (5 letters +
\0)
```

Method 3: Character by character

C

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Created by Pawan Soni

3. String Input & Output

Common Ways:

Function	Use	Problem / Warning

THE ARTIFICIAL EDUCATION

scanf("%s", str)	Word input	Space ke baad nahi padta
gets(str)	Full line input	**Unsafe** (buffer overflow)
fgets(str, size, stdin)	Safe full line input	**Best method**
printf("%s", str)	Print string	Safe
puts(str)	Print string + new line	Safe

Best Practice (Recommended):

C

```
char str[100];
fgets(str, 100, stdin);     // Safe way
```

4. <string.h> Header Functions (Sabse Important)

Yeh header include karna padta hai:

C

```
#include <string.h>
```

Function	Kaam	Example
strlen(str)	Length return karta hai (\0 count nahi hota)	strlen("Hello") → 5
strcpy(dest, src)	Source ko destination mein copy	strcpy(s2, s1);
strcat(dest, src)	Source ko destination ke end mein jodta hai	strcat(s1, s2);
strcmp(s1, s2)	Compare karta hai (0 = equal)	strcmp("abc", "abc") → 0
strrev(str)	String reverse karta hai (Turbo C mein)	-
strlwr(str)	Lowercase mein convert	-
strupr(str)	Uppercase mein convert	-

Note: strrev, strlwr, strupr sirf Turbo C mein kaam karte hain.

5. Practical Examples (String Functions)

Example 1: Length using strlen

C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[50] = "College Wallah";
    printf("Length = %zu", strlen(str));   // %zu for size_t
    return 0;
}
```

C Language Complete Notes
Learn C programming from basics to advanced
Created by Pawan Soni
learn computer, control your future

Example 2: Copy & Concatenate

C

```
char s1[50] = "Hello";
char s2[50] = " World";
strcpy(s1, "Good Morning");     // s1 ab "Good Morning" ho gaya
strcat(s1, s2);                 // s1 = "Good Morning World"
```

Example 3: Compare two strings

C

```
if(strcmp(s1, s2) == 0)
    printf("Dono strings same hain");
```

6. Manual Implementation (Without string.h)

Length without strlen:

C

```
int length(char str[]) {
    int i = 0;
    while(str[i] != '\0') i++;
    return i;
}
```

Copy without strcpy:

C

```
void myStrcpy(char dest[], char src[]) {
    int i = 0;
    while(src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}
```

7. 2D Array of Strings

C

```
char names[5][20] = {
    "Pawan",
    "Rahul",
    "Aman",
    "Priya",
    "Sneha"
};
```

Print karna:

C

Complete Notes
Learn C programming from basics to advanced with real world examples
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
for(int i = 0; i < 5; i++) {
    printf("%s\n", names[i]);
}
```

8. Pointers with Strings

C

```
char *ptr = "Hello";     // Pointer to string literal
printf("%s", ptr);
```

✅ PRACTICE SET 8

Q1. User se string le aur uski length print karo (without strlen).

C

```
#include <stdio.h>
int main() {
    char str[100];
    int i = 0;

    printf("String daalo: ");
    fgets(str, 100, stdin);   // safe input

    // Manual length calculate
    while(str[i] != '\0' && str[i] != '\n') {
        i++;
    }

    printf("Length = %d", i);
    return 0;
}
```

Sample Input: Hello World **Output:** Length = 11

Q2. String ko reverse karo (without strrev).

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    scanf("%[^\n]s", str);   // full line with spaces

    int len = 0;
    while(str[len] != '\0') len++;

    // Reverse using two pointers
    for(int i = 0; i < len/2; i++) {
```

```
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }

    printf("Reversed String: %s", str);
    return 0;
}
```

Sample Input: Pawan **Output:** Reversed String: nawaP

Q3. String palindrome hai ya nahi check karo.

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    scanf("%[^\n]s", str);

    int len = 0;
    while(str[len] != '\0') len++;

    int isPalindrome = 1;
    for(int i = 0; i < len/2; i++) {
        if(str[i] != str[len-1-i]) {
            isPalindrome = 0;
            break;
        }
    }

    if(isPalindrome)
        printf("Palindrome hai");
    else
        printf("Palindrome nahi hai");
    return 0;
}
```

Sample Input: madam → **Palindrome hai Sample Input:** hello → **Palindrome nahi hai**

Q4. String mein vowels count karo.

C

```
#include <stdio.h>
int main() {
    char str[100];
```

```
    int count = 0;
    printf("String daalo: ");
    fgets(str, 100, stdin);

    for(int i = 0; str[i] != '\0'; i++) {
        char ch = str[i];
        if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
           ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
            count++;
    }
    printf("Vowels ki sankhya = %d", count);
    return 0;
}
```

Q5. Do strings ko compare karo (without strcmp).

C

```
#include <stdio.h>
int main() {
    char s1[100], s2[100];
    printf("Pehli string daalo: ");
    scanf("%[^\n]s", s1);
    printf("Doosri string daalo: ");
    scanf(" %[^\n]s", s2);

    int i = 0;
    while(s1[i] == s2[i] && s1[i] != '\0' && s2[i] != '\0') i++;

    if(s1[i] == '\0' && s2[i] == '\0')
        printf("Dono strings same hain");
    else
        printf("Dono strings alag hain");
    return 0;
}
```

Q6. String ko uppercase mein convert karo.

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    fgets(str, 100, stdin);
```

```
    for(int i = 0; str[i] != '\0'; i++) {
        if(str[i] >= 'a' && str[i] <= 'z')
            str[i] = str[i] - 32;   // ASCII difference
    }
    printf("Uppercase: %s", str);
    return 0;
}
```

Q7. String mein sabse zyada baar aane wala character dhundo.

C

```
#include <stdio.h>
int main() {
    char str[100];
    int freq[256] = {0};   // ASCII range
    printf("String daalo: ");
    fgets(str, 100, stdin);

    for(int i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }

    char maxChar = '\0';
    int maxCount = 0;
    for(int i = 0; i < 256; i++) {
        if(freq[i] > maxCount) {
            maxCount = freq[i];
            maxChar = (char)i;
        }
    }
    printf("Sabse zyada baar aane wala character = '%c' (%d baar)", maxChar,
maxCount);
    return 0;
}
```

learn computer, control your future

Q8. String ke words count karo.

C

```
#include <stdio.h>
int main() {
    char str[200];
    int words = 0, i = 0;
    printf("String daalo: ");
    fgets(str, 200, stdin);
```

THE ARTIFICIAL EDUCATION

```
    while(str[i] != '\0') {
        if(str[i] == ' ' && str[i+1] != ' ' && str[i+1] != '\0')
            words++;
        i++;
    }
    words++;   // last word count
    printf("Total words = %d", words);
    return 0;
}
```

Q9. String ko title case mein convert karo (har word ka pehla letter capital).

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    fgets(str, 100, stdin);

    int capitalize = 1;
    for(int i = 0; str[i] != '\0'; i++) {
        if(str[i] == ' ' || str[i] == '\n')
            capitalize = 1;
        else if(capitalize && str[i] >= 'a' && str[i] <= 'z') {
            str[i] = str[i] - 32;
            capitalize = 0;
        } else if(str[i] >= 'A' && str[i] <= 'Z')
            capitalize = 0;
    }
    printf("Title Case: %s", str);
    return 0;
}
```

Q10. User se multiple lines input lo jab tak "end" nahi likha jaaye.

C

```
#include <stdio.h>
int main() {
    char line[200];
    printf("Lines daalo (end likhkar stop karo):\n");

    while(1) {
        fgets(line, 200, stdin);
```

```
        if(line[0] == 'e' && line[1] == 'n' && line[2] == 'd' && line[3] == '\n')
            break;
        printf("Aapne likha: %s", line);
    }
    printf("Program band ho gaya!");
    return 0;
}
```

Q11. String mein substring dhundo.

C

```
#include <stdio.h>
int main() {
    char str[100], sub[50];
    printf("Main string daalo: ");
    scanf("%[^\n]s", str);
    printf("Substring dhundo: ");
    scanf(" %[^\n]s", sub);

    int found = 0;
    for(int i = 0; str[i] != '\0'; i++) {
        int j = 0;
        while(sub[j] != '\0' && str[i+j] == sub[j]) j++;
        if(sub[j] == '\0') {
            found = 1;
            printf("Substring index %d par mila", i);
            break;
        }
    }
    if(!found) printf("Substring nahi mila");
    return 0;
}
```

Q12. Remove all spaces from string.

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    fgets(str, 100, stdin);

    int j = 0;
    for(int i = 0; str[i] != '\0'; i++) {
```

```
        if(str[i] != ' ' && str[i] != '\n')
            str[j++] = str[i];
    }
    str[j] = '\0';
    printf("Spaces hatane ke baad: %s", str);
    return 0;
}
```

Q13. String ko sort karo (alphabetical order).

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    scanf("%s", str);

    int n = 0;
    while(str[n] != '\0') n++;

    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(str[j] > str[j+1]) {
                char temp = str[j];
                str[j] = str[j+1];
                str[j+1] = temp;
            }
        }
    }
    printf("Sorted String: %s", str);
    return 0;
}
```

Q14. Anagram check karo (do strings ke characters same hain ya nahi).

C

```
#include <stdio.h>
int main() {
    char s1[100], s2[100];
    int count[256] = {0};

    printf("Pehli string: ");
    scanf("%s", s1);
    printf("Doosri string: ");
```

```
    scanf("%s", s2);

    int i = 0;
    while(s1[i] != '\0') count[(unsigned char)s1[i++]]++;
    i = 0;
    while(s2[i] != '\0') count[(unsigned char)s2[i++]]--;

    int isAnagram = 1;
    for(int j = 0; j < 256; j++) {
        if(count[j] != 0) {
            isAnagram = 0;
            break;
        }
    }
    if(isAnagram)
        printf("Anagram hai");
    else
        printf("Anagram nahi hai");
    return 0;
}
```

Q15. String ke har character ko next character se replace karo (Caesar cipher simple version).

C

```
#include <stdio.h>
int main() {
    char str[100];
    printf("String daalo: ");
    fgets(str, 100, stdin);

    for(int i = 0; str[i] != '\0'; i++) {
        if(str[i] >= 'a' && str[i] <= 'z')
            str[i] = (str[i] == 'z') ? 'a' : str[i] + 1;
        else if(str[i] >= 'A' && str[i] <= 'Z')
            str[i] = (str[i] == 'Z') ? 'A' : str[i] + 1;
    }
    printf("Caesar Cipher: %s", str);
    return 0;
}
```

✅ STRUCTURES IN C

Structures C language ka bahut powerful feature hai. Yeh allow karta hai ki hum **alag-alag data types** ke variables ko **ek saath** group mein rakh sakein.

THE ARTIFICIAL EDUCATION

Jaise ek student ka record: naam (string), roll number (int), marks (float), grade (char) — sab ek hi jagah rakh sakte hain.

1. Structure Kya Hai?

- Structure ek **user-defined data type** hai.
- Isme different data types ke variables ko ek naam ke neeche rakha jaata hai.
- Normal array mein sirf same type ke data rakh sakte hain, lekin structure mein mix type rakh sakte hain.

Real Life Example: Ek student ka record:

- Name → string
- Roll No → int
- Marks → float
- Grade → char

Sabko alag-alag variables mein rakhna mushkil hai. Structure mein ek hi variable mein sab rakh sakte hain.

2. Syntax of Structure

C

```
struct structure_name {
    data_type member1;
    data_type member2;
    data_type member3;
    ...
};
```

Example:

C

```
struct Student {
    char name[50];
    int roll;
    float marks;
    char grade;
};
```

Note: Structure declaration ke end mein **semicolon (;)** lagana zaroori hai.

3. Structure Variable Create Karna

C

```
struct Student s1;          // s1 ek Student type ka variable hai
struct Student s2, s3;      // multiple variables
```

Full Example:

C

```
#include <stdio.h>
```

```c
struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1;          // structure variable
    return 0;
}
```

4. Accessing Structure Members (Dot Operator .)

Structure ke members ko access karne ke liye **. (dot operator)** use karte hain.

C

```c
s1.roll = 101;
s1.marks = 92.5;
```

Complete Working Example:

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1;

    printf("Enter name: ");
    scanf("%s", s1.name);
    printf("Enter roll: ");
    scanf("%d", &s1.roll);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    printf("\n--- Student Details ---\n");
    printf("Name  : %s\n", s1.name);
    printf("Roll  : %d\n", s1.roll);
    printf("Marks : %.2f\n", s1.marks);
```

Learn C programming from basics to advanced with real world examples
THE ARTIFICIAL EDUCATION

```
    return 0;
}
```

5. Initialization of Structure

C

```
struct Student s1 = {"Pawan", 101, 95.5};
struct Student s2 = {.roll = 102, .marks = 88.0};   // designated initialization
```

6. Array of Structures

Bahut important concept!

C

```
struct Student class[50];     // 50 students ka record
```

Example:

C

```
struct Student class[3] = {
    {"Rahul", 1, 85.5},
    {"Priya", 2, 92.0},
    {"Aman",  3, 78.5}
};
```

7. Pointer to Structure

C

```
struct Student *ptr;
ptr = &s1;

printf("%s", ptr->name);     // arrow operator (->) use karte hain
```

Note: ptr->member = (*ptr).member dono same hain.

8. Nested Structure (Structure inside Structure)

C

```
struct Date {
    int day, month, year;
};

struct Student {
    char name[50];
    int roll;
    struct Date dob;        // nested structure
};
```

9. Passing Structure to Function

Call by Value:

C

```
void printStudent(struct Student s) { ... }
```

Call by Reference (Recommended):

C

```
void printStudent(struct Student *s) {
    printf("%s", s->name);
}
```

10. typedef with Structure (Code Short Karne Ke Liye)

C

```
typedef struct Student {
    char name[50];
    int roll;
    float marks;
} Stu;

Stu s1;     // ab "struct Student" likhne ki zarurat nahi
```

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ PRACTICE SET 9

Q1. Ek structure banao Book ke liye (title, author, price) aur user se input lekar print karo.

C

```c
#include <stdio.h>

struct Book {
    char title[100];
    char author[50];
    float price;
};

int main() {
    struct Book b;
    printf("Book title: ");
    scanf(" %[^\n]s", b.title);
    printf("Author: ");
    scanf(" %[^\n]s", b.author);
    printf("Price: ");
    scanf("%f", &b.price);

    printf("\nBook Details:\nTitle: %s\nAuthor: %s\nPrice: %.2f", b.title,
b.author, b.price);
    return 0;
}
```

Q2. Array of Structures banao aur 3 students ka data input lekar print karo.

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s[3];

    for(int i = 0; i < 3; i++) {
        printf("\nStudent %d ka naam: ", i+1);
        scanf("%s", s[i].name);
        printf("Roll Number: ");
```

```
        scanf("%d", &s[i].roll);
        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }

    printf("\n--- Students Details ---\n");
    for(int i = 0; i < 3; i++) {
        printf("Name: %s | Roll: %d | Marks: %.2f\n",
               s[i].name, s[i].roll, s[i].marks);
    }
    return 0;
}
```

Sample Output:

text

```
Student 1 ka naam: Rahul
Roll Number: 101
Marks: 92.5

Student 2 ka naam: Priya
Roll Number: 102
Marks: 88.0

Student 3 ka naam: Aman
Roll Number: 103
Marks: 95.0

--- Students Details ---
Name: Rahul | Roll: 101 | Marks: 92.50
Name: Priya | Roll: 102 | Marks: 88.00
Name: Aman  | Roll: 103 | Marks: 95.00
```

Q3. Structure pointer use karke student ka data print karo.

C

Created by Pawan Soni

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
```

```
    struct Student s;
    struct Student *ptr = &s;   // Pointer to structure

    printf("Name daalo: ");
    scanf("%s", ptr->name);
    printf("Roll: ");
    scanf("%d", &ptr->roll);
    printf("Marks: ");
    scanf("%f", &ptr->marks);

    printf("\n--- Student Details (using Pointer) ---\n");
    printf("Name: %s\nRoll: %d\nMarks: %.2f\n",
           ptr->name, ptr->roll, ptr->marks);
    return 0;
}
```

Explanation: -> (arrow operator) pointer ke through members access karne ke liye use hota hai.

Q4. Nested structure banao Employee ke andar Date of Joining.

C

```
#include <stdio.h>

struct Date {
    int day, month, year;
};

struct Employee {
    char name[50];
    int id;
    struct Date doj;   // Nested Structure
};

int main() {
    struct Employee e;

    printf("Employee Name: ");
    scanf("%s", e.name);
    printf("Employee ID: ");
    scanf("%d", &e.id);
    printf("Joining Date (DD MM YYYY): ");
    scanf("%d %d %d", &e.doj.day, &e.doj.month, &e.doj.year);

    printf("\nEmployee Details:\n");
```

```
    printf("Name: %s\nID: %d\nJoining Date: %02d/%02d/%d\n",
           e.name, e.id, e.doj.day, e.doj.month, e.doj.year);
    return 0;
}
```

Q5. Function mein structure pass karke employee ka salary double kar do.

C

```c
#include <stdio.h>

struct Employee {
    char name[50];
    float salary;
};

void doubleSalary(struct Employee *e) {   // Pointer pass kar rahe hain
    e->salary = e->salary * 2;
}

int main() {
    struct Employee e;
    printf("Name daalo: ");
    scanf("%s", e.name);
    printf("Salary daalo: ");
    scanf("%f", &e.salary);

    doubleSalary(&e);

    printf("New Salary of %s = %.2f", e.name, e.salary);
    return 0;
}
```

Explanation: Structure ko pointer se pass karne se original value change ho jaati hai.

Q6. Structure ke members ko compare karo (do employees mein kaun zyada marks wala hai).

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    float marks;
};

int main() {
```

```c
    struct Student s1, s2;

    printf("Student 1 - Name & Marks: ");
    scanf("%s %f", s1.name, &s1.marks);
    printf("Student 2 - Name & Marks: ");
    scanf("%s %f", s2.name, &s2.marks);

    if(s1.marks > s2.marks)
        printf("%s ke marks zyada hain", s1.name);
    else if(s2.marks > s1.marks)
        printf("%s ke marks zyada hain", s2.name);
    else
        printf("Dono ke marks barabar hain");
    return 0;
}
```

Q7. typedef use karke short naam se structure use karo.

C

```c
#include <stdio.h>

typedef struct Student {
    char name[50];
    int roll;
    float marks;
} Stu;     // Ab "Stu" short naam ban gaya

int main() {
    Stu s1 = {"Rahul", 101, 92.5};   // struct Student nahi likhna pada
    printf("Name: %s | Roll: %d | Marks: %.2f", s1.name, s1.roll, s1.marks);
    return 0;
}
```

Created by Pawan Soni

Q8. Structure array sort karo (marks ke hisaab se).

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    float marks;
};

int main() {
```

```c
    struct Student s[3] = {{"Rahul", 85}, {"Priya", 92}, {"Aman", 78}};

    // Bubble Sort by marks
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2-i; j++) {
            if(s[j].marks < s[j+1].marks) {
                struct Student temp = s[j];
                s[j] = s[j+1];
                s[j+1] = temp;
            }
        }
    }

    printf("Marks ke hisaab se sorted list:\n");
    for(int i = 0; i < 3; i++) {
        printf("%s - %.2f\n", s[i].name, s[i].marks);
    }
    return 0;
}
```

Q9. Structure mein string member ko reverse karo.

C

```c
#include <stdio.h>

struct Student {
    char name[50];
};

int main() {
    struct Student s;
    printf("Name daalo: ");
    scanf("%s", s.name);

    int len = 0;
    while(s.name[len] != '\0') len++;

    for(int i = 0; i < len/2; i++) {
        char temp = s.name[i];
        s.name[i] = s.name[len-1-i];
        s.name[len-1-i] = temp;
    }
```

```
    printf("Reversed Name: %s", s.name);
    return 0;
}
```

Q10. Menu driven program banao jismein structure operations ho (add student, show all, search by roll).

C

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s[10];
    int count = 0, choice, r;

    while(1) {
        printf("\n1. Add Student\n2. Show All\n3. Search by Roll\n4.
Exit\nChoice: ");
        scanf("%d", &choice);

        if(choice == 4) break;

        if(choice == 1 && count < 10) {
            printf("Name: ");
            scanf("%s", s[count].name);
            printf("Roll: ");
            scanf("%d", &s[count].roll);
            printf("Marks: ");
            scanf("%f", &s[count].marks);
            count++;
        }
        else if(choice == 2) {
            for(int i = 0; i < count; i++) {
                printf("%s | Roll:%d | Marks:%.2f\n", s[i].name, s[i].roll,
s[i].marks);
            }
        }
        else if(choice == 3) {
```

```
            printf("Roll Number daalo: ");
            scanf("%d", &r);
            for(int i = 0; i < count; i++) {
                if(s[i].roll == r) {
                    printf("Found: %s | Marks: %.2f\n", s[i].name, s[i].marks);
                    break;
                }
            }
        }
    }
    return 0;
}
```

Q11. 2 students ke records ko compare karo (same roll number hai ya nahi).

C

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
};

int main() {
    struct Student s1, s2;

    printf("Student 1 - Name & Roll: ");
    scanf("%s %d", s1.name, &s1.roll);
    printf("Student 2 - Name & Roll: ");
    scanf("%s %d", s2.name, &s2.roll);

    if(s1.roll == s2.roll)
        printf("Dono students ka roll number same hai");
    else
        printf("Roll number alag hai");
    return 0;
}
```

Q12. Structure ka size print karo (sizeof operator use karke).

C

```
#include <stdio.h>

struct Student {
```

```
    char name[50];
    int roll;
    float marks;
    char grade;
};

int main() {
    printf("Size of structure Student = %zu bytes", sizeof(struct Student));
    return 0;
}
```

✓ FILE I/O 10

with real-world examples

1. File I/O Kya Hai?

Jab tum program chalate ho, sab data **RAM** (computer ki temporary memory) mein rehta hai. Program band karte hi yeh data **khataam** ho jaata hai.

File I/O ka matlab hai: Data ko **Hard Disk** (permanent storage) mein save karna, taaki program band hone ke baad bhi data safe rahe.

Real Life Example:

- Tum notes likhte ho phone mein → agar phone off kar do toh notes gayab ho jaate hain? Nahi, kyunki woh **file** mein save hote hain.
- C program mein bhi agar tum data file mein save nahi karoge, toh program band karte hi data gayab ho jaayega.

2. File Handling Ke 3 Basic Steps

Har baar file ke saath yeh teen kaam karna padta hai:

1. **File Open karo** → Computer ko batao ki file kaam karna hai.
2. **Read ya Write karo** → Data padho ya likho.
3. **File Close karo** → Kaam khatam hone ke baad file band kar do (important hai!).

3. File Modes (Sabse Important)

Created by **Pawan Soni**

File open karte waqt mode batana padta hai:

Mode	Matlab (Simple Language)	Kab Use Karo?
"r"	Sirf padhne ke liye (Read)	Purani file se data padhna hai
"w"	Likhne ke liye (purani file delete ho jaayegi)	Nayi file banani hai ya purani overwrite
"a"	End mein add karne ke liye (Append)	Purani file mein naya data jodna
"r+"	Padh bhi sakte ho aur likh bhi sakte ho	Purani file mein changes karne hain
"w+"	Likh sakte ho aur padh bhi sakte ho (overwrite)	Nayi file banaani hai

THE ARTIFICIAL EDUCATION

Sabse commonly use hone wale:

- "r" → padhne ke liye
- "w" → nayi file banane ya purani delete karne ke liye
- "a" → purani file mein naya data add karne ke liye

4. File Pointer Kya Hai?

FILE *fp; Yeh ek special variable hai jo file ko point karta hai (jaise remote control TV ko point karta hai).

Example:

C

```
FILE *fp;                    // File pointer banaya
fp = fopen("myfile.txt", "w");   // File kholi
```

5. Step-by-Step Practical Example

Program: Ek file banao aur usme data likho

C

```
#include <stdio.h>

int main() {
    FILE *fp;                           // Step 1: File pointer banaya

    fp = fopen("student.txt", "w");     // Step 2: File kholi (write mode)

    if(fp == NULL) {                    // Agar file nahi khuli toh error
        printf("File kholne mein problem aayi!");
        return 1;
    }

    // Step 3: File mein likho
    fprintf(fp, "Name: Pawan\n");
    fprintf(fp, "Roll No: 101\n");
    fprintf(fp, "Marks: 95.5\n");

    fclose(fp);                         // Step 4: File band kar di

    printf("Data file mein save ho gaya!");
    return 0;
}
```

Output: Data file mein save ho gaya! Aur folder mein ek file student.txt ban jaayegi jismein upar wala data hoga.

6. File Se Data Padhna (Reading)

C

```c
#include <stdio.h>
int main() {
    FILE *fp;
    char ch;

    fp = fopen("student.txt", "r");    // Read mode mein khola

    if(fp == NULL) {
        printf("File nahi mili!");
        return 1;
    }

    while((ch = fgetc(fp)) != EOF) {   // EOF = End Of File
        printf("%c", ch);
    }

    fclose(fp);
    return 0;
}
```

7. File Mein Data Likhna (Writing)

Sabse common function: fprintf

C

```c
fprintf(fp, "Jo likhna hai\n");
```

Poora Example:

C

```c
#include <stdio.h>
int main() {
    FILE *fp;

    fp = fopen("mydata.txt", "w");     // write mode (purani file delete ho
jayegi)

    if(fp == NULL) {
        printf("File nahi khul saki!");
        return 1;
    }

    fprintf(fp, "Name: Pawan\n");
    fprintf(fp, "Age: 22\n");
    fprintf(fp, "Marks: 95.75\n");
```

```
    fprintf(fp, "Grade: A\n");

    fclose(fp);                         // file band karna zaroori hai
    printf("Data file mein save ho gaya!");
    return 0;
}
```

Output: Ek file mydata.txt ban jaayegi jismein upar wala text hoga.

8. File Se Data Padhna (Reading)

3 main tareeke hain:

1. **fgetc** → ek-ek character padhta hai
2. **fgets** → poori line padhta hai (sabse safe aur best)
3. **fscanf** → formatted way mein padhta hai

Best Method (fgets use karke):

C

```
#include <stdio.h>
int main() {
    FILE *fp;
    char line[100];

    fp = fopen("mydata.txt", "r");

    if(fp == NULL) {
        printf("File nahi mili!");
        return 1;
    }

    while(fgets(line, 100, fp) != NULL) {   // jab tak end nahi hota
        printf("%s", line);
    }

    fclose(fp);
    return 0;
}
```

Yeh pura file content line-by-line print kar dega.

9. File Mein Naya Data Add Karna (Append Mode)

C

```
fp = fopen("mydata.txt", "a");     // "a" = append mode
```

Example:

C

```
fprintf(fp, "New Line added\n");
```

✅ PRACTICE SET 11

Q1. User se naam, age aur marks le aur file mein save karo.

C

```c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float marks;

    printf("Naam daalo: ");
    scanf("%s", name);
    printf("Age daalo: ");
    scanf("%d", &age);
    printf("Marks daalo: ");
    scanf("%f", &marks);

    FILE *fp = fopen("student.txt", "w");
    if(fp == NULL) {
        printf("File nahi khul saki!");
        return 1;
    }

    fprintf(fp, "Name: %s\n", name);
    fprintf(fp, "Age: %d\n", age);
    fprintf(fp, "Marks: %.2f\n", marks);

    fclose(fp);
    printf("Data file 'student.txt' mein save ho gaya!");
    return 0;
}
```

Sample Output:

text

```text
Naam daalo: Pawan
Age daalo: 22
Marks daalo: 95.5
Data file 'student.txt' mein save ho gaya!
```

Q2. File se data padhkar screen pe print karo.

C

Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("student.txt", "r");
    if(fp == NULL) {
        printf("File nahi mili!");
        return 1;
    }

    char ch;
    while((ch = fgetc(fp)) != EOF) {
        printf("%c", ch);
    }

    fclose(fp);
    return 0;
}
```

Output: (jo data Q1 mein save kiya tha woh print hoga)

Q3. File mein multiple lines likho aur fgets se line-by-line padho.

C

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("notes.txt", "w");
    fprintf(fp, "Line 1: Hello World\n");
    fprintf(fp, "Line 2: C Programming\n");
    fprintf(fp, "Line 3: File I/O Practice\n");
    fclose(fp);

    // Ab padhte hain line by line
    fp = fopen("notes.txt", "r");
    char line[100];

    printf("File content line by line:\n");
    while(fgets(line, 100, fp) != NULL) {
        printf("%s", line);
    }

    fclose(fp);
    return 0;
}
```

Output:

text

```
File content line by line:
Line 1: Hello World
Line 2: C Programming
Line 3: File I/O Practice
```

Complete Notes

Q4. File ka size calculate karo (ftell use karke).

C

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("student.txt", "r");
    if(fp == NULL) {
        printf("File nahi mili!");
        return 1;
    }

    fseek(fp, 0, SEEK_END);        // end tak le jaao
    long size = ftell(fp);         // current position = file size

    printf("File ka size = %ld bytes", size);

    fclose(fp);
    return 0;
}
```

Q5. Ek file ko dusri file mein copy karo.

C

```c
#include <stdio.h>

int main() {
    FILE *source = fopen("student.txt", "r");
    FILE *dest = fopen("copy.txt", "w");

    if(source == NULL || dest == NULL) {
        printf("File error!");
        return 1;
    }

    char ch;
    while((ch = fgetc(source)) != EOF) {
```

```
        fputc(ch, dest);
    }

    fclose(source);
    fclose(dest);
    printf("File successfully copied to copy.txt");
    return 0;
}
```

Q6. File mein word count karo.

C

Learn C programming from basics to advanced

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("student.txt", "r");
    if(fp == NULL) {
        printf("File nahi mili!");
        return 1;
    }

    int words = 0;
    char ch;
    int inWord = 0;

    while((ch = fgetc(fp)) != EOF) {
        if(ch == ' ' || ch == '\n' || ch == '\t') {
            inWord = 0;
        } else if(inWord == 0) {
            inWord = 1;
            words++;
        }
    }

    fclose(fp);
    printf("Total words in file = %d", words);
    return 0;
}
```

Q7. Structure use karke student ka record file mein save karo.

C

```
#include <stdio.h>
```

```c
struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s;
    printf("Name: ");
    scanf("%s", s.name);
    printf("Roll: ");
    scanf("%d", &s.roll);
    printf("Marks: ");
    scanf("%f", &s.marks);

    FILE *fp = fopen("students.dat", "wb");   // binary mode
    fwrite(&s, sizeof(s), 1, fp);
    fclose(fp);

    printf("Student record file mein save ho gaya!");
    return 0;
}
```

Q8. Binary file banao aur usme integer array save karo.

C

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    FILE *fp = fopen("data.bin", "wb");
    fwrite(arr, sizeof(int), 5, fp);
    fclose(fp);

    printf("Integer array binary file mein save ho gaya!");
    return 0;
}
```

Q9. Text file se specific word dhundo.

C

```c
#include <stdio.h>
#include <string.h>
```

```c
int main() {
    FILE *fp = fopen("student.txt", "r");
    char word[50], search[50];
    int found = 0;

    printf("Dhundne wala word daalo: ");
    scanf("%s", search);

    while(fscanf(fp, "%s", word) == 1) {
        if(strcmp(word, search) == 0) {
            found = 1;
            break;
        }
    }

    if(found)
        printf("Word mila!");
    else
        printf("Word nahi mila!");

    fclose(fp);
    return 0;
}
```

Q10. File ka content reverse karke nai file mein save karo.

C

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("student.txt", "r");
    FILE *newfp = fopen("reversed.txt", "w");

    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);

    for(long i = size - 2; i >= 0; i--) {   // -2 last \n ko ignore karne ke liye
        fseek(fp, i, SEEK_SET);
        char ch = fgetc(fp);
        fputc(ch, newfp);
    }
```

```
    fclose(fp);
    fclose(newfp);
    printf("File ka content reverse karke 'reversed.txt' mein save kar diya gaya!");
    return 0;
}
```

✅ DYNAMIC MEMORY ALLOCATION IN C –

Bhai, ab hum **Dynamic Memory Allocation** cover kar rahe hain. Yeh C language ka bahut important aur powerful topic hai.

1. Dynamic Memory Allocation Kya Hai?

- **Static Memory** → Program shuru hone se pehle hi memory fix ho jaati hai (jaise int arr[100];).
- **Dynamic Memory** → Program chalte waqt (runtime) mein jitni memory chahiye, utni memory allocate (banwa) sakte ho.

Simple Language Mein: Jaise ghar mein pehle se hi 5 almirah hain (static) → agar aur chahiye toh naya almirah kharidna padega (dynamic).
Dynamic memory **Heap** memory se aati hai (RAM ka ek hissa).

2. Kyun Zarurat Padti Hai?

- Jab size pehle se pata nahi hota (user input ke hisaab se array banani ho)
- Jab program chalte waqt memory ki zarurat badal jaati hai
- Memory ko efficiently use karne ke liye
- Linked List, Tree, Graph jaise data structures banane ke liye

3. 4 Important Functions (Sabse Important)

Yeh sab functions <stdlib.h> header se aate hain.

C

```
#include <stdlib.h>
```

Function	Kaam	Returns	Kab Use Karo?
malloc()	Memory allocate karta hai	void*	Sirf size chahiye
calloc()	Memory allocate + sabko 0 se initialize	void*	Array banani ho aur 0 se shuru
realloc()	Pehle allocate kiye memory ka size badhao	void*	Memory ko resize karna ho
free()	Allocate kiye memory ko release karta hai	void	Memory leak rokne ke liye

Complete Notes
Learn C programming from basics to advanced
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

4. Detailed Explanation with Examples

1. malloc() – Memory Allocate Karo

C

```
int *ptr = (int*)malloc(5 * sizeof(int));
```

- 5 integers ke liye memory allocate karta hai
- sizeof(int) = ek integer kitni memory leta hai (usually 4 bytes)

Full Example:

C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Kitne numbers chahiye? ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));

    if(arr == NULL) {
        printf("Memory nahi mili!");
        return 1;
    }

    for(int i = 0; i < n; i++) {
        printf("Enter number %d: ", i+1);
        scanf("%d", &arr[i]);
    }

    printf("Numbers: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);        // Memory release kar di
    return 0;
}
```

2. calloc() – Memory Allocate + Zero Initialize

C

```
int *arr = (int*)calloc(5, sizeof(int));
```

- malloc se farak: sab values ko 0 se initialize kar deta hai

Example:

C Language Complete Notes
THE ARTIFICIAL EDUCATION

C

```c
int *arr = (int*)calloc(5, sizeof(int));
for(int i = 0; i < 5; i++) {
    printf("%d ", arr[i]);   // sab 0 print honge
}
```

3. realloc() – Memory Ka Size Badhao

C

```c
ptr = (int*)realloc(ptr, new_size);
```

Example:

C

```c
int *arr = (int*)malloc(5 * sizeof(int));
// ... kuch kaam kiya

arr = (int*)realloc(arr, 10 * sizeof(int));   // ab 10 numbers store kar sakta hai
```

4. free() – Memory Release Karo

C

```c
free(ptr);
```

Zaroori Hai: Agar free() nahi karoge toh **Memory Leak** ho jaayega (program memory khatam kar deta hai).

5. Common Mistakes (Yaad Rakhna)

1. malloc/calloc ke baad NULL check karna bhool jaana
2. free() karna bhool jaana → Memory Leak
3. free() karne ke baad pointer use karna → Dangling Pointer
4. realloc ke baad purane pointer ko use karna

✅ PRACTICE SET 12

Q1. Ek integer ke liye dynamic memory allocate karo aur value store karke print karo.

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int*)malloc(sizeof(int));   // 1 integer ke liye memory
    if(ptr == NULL) {
        printf("Memory nahi mili!");
        return 1;
    }
    *ptr = 100;
```

Complete Notes
Learn C programming from basics to advanced
Created by Pawan Soni

```
    printf("Value = %d", *ptr);
    free(ptr);          // memory release
    return 0;
}
```

Output: Value = 100

Q2. User se size le aur utne numbers ka dynamic array banao (malloc use karke).

C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Kitne numbers chahiye? ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));
    if(arr == NULL) {
        printf("Memory nahi mili!");
        return 1;
    }

    for(int i = 0; i < n; i++) {
        printf("Number %d: ", i+1);
        scanf("%d", &arr[i]);
    }

    printf("Array: ");
    for(int i = 0; i < n; i++) printf("%d ", arr[i]);

    free(arr);
    return 0;
}
```

Q3. calloc use karke array banao aur check karo ki sab values 0 hain ya nahi.

C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int*)calloc(n, sizeof(int));   // calloc automatically 0 karta
```

```
hai

    printf("Calloc se banaya array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    free(arr);
    return 0;
}
```

Output: Calloc se banaya array: 0 0 0 0 0

Learn C programming from basics to advanced
with real-world examples

Q4. realloc use karke array ka size badhao (pehle 3 elements, baad mein 6).

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(3 * sizeof(int));
    arr[0] = 10; arr[1] = 20; arr[2] = 30;

    arr = (int*)realloc(arr, 6 * sizeof(int));   // size badhaya
    arr[3] = 40; arr[4] = 50; arr[5] = 60;

    for(int i = 0; i < 6; i++) printf("%d ", arr[i]);
    free(arr);
    return 0;
}
```

Output: 10 20 30 40 50 60

Q5. Memory leak rokne ke liye free() ka sahi use dikhao.

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int*)malloc(10 * sizeof(int));
    // ... kuch kaam kiya

    free(ptr);            // memory release
    ptr = NULL;           // dangling pointer se bachne ke liye

    printf("Memory release ho gayi!");
```

```
    return 0;
}
```

Q6. Dynamic 2D array banao (user se rows aur columns leke).

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows, cols;
    printf("Rows aur Columns daalo: ");
    scanf("%d %d", &rows, &cols);

    int **arr = (int**)malloc(rows * sizeof(int*));
    for(int i = 0; i < rows; i++) {
        arr[i] = (int*)malloc(cols * sizeof(int));
    }

    // values daalo
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            arr[i][j] = i + j;
        }
    }

    printf("2D Array:\n");
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    // memory release
    for(int i = 0; i < rows; i++) free(arr[i]);
    free(arr);
    return 0;
}
```

Q7. Dynamic string input lo (user jitni bhi lambi string de sake).

C

C Language
THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *str = NULL;
    size_t size = 0;
    printf("String daalo (jitni bhi lambi): ");
    getline(&str, &size, stdin);   // dynamic memory automatically manage karta
hai

    printf("Aapki string: %s", str);
    free(str);
    return 0;
}
```

Q8. Structure ke andar dynamic array banao (student ke multiple subjects ke marks).

C

```c
#include <stdio.h>
#include <stdlib.h>

struct Student {
    char name[50];
    int *marks;      // dynamic array
    int subjects;
};

int main() {
    struct Student s;
    printf("Name: ");
    scanf("%s", s.name);
    printf("Kitne subjects? ");
    scanf("%d", &s.subjects);

    s.marks = (int*)malloc(s.subjects * sizeof(int));

    for(int i = 0; i < s.subjects; i++) {
        printf("Subject %d marks: ", i+1);
        scanf("%d", &s.marks[i]);
    }

    printf("\n%s ke marks: ", s.name);
    for(int i = 0; i < s.subjects; i++) printf("%d ", s.marks[i]);
```

```
    free(s.marks);
    return 0;
}
```

Q9. realloc use karke dynamic array ko resize karo aur purane data ko preserve rakho.

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(3 * sizeof(int));
    arr[0] = 10; arr[1] = 20; arr[2] = 30;

    arr = (int*)realloc(arr, 5 * sizeof(int));   // size badhaya
    arr[3] = 40;
    arr[4] = 50;

    for(int i = 0; i < 5; i++) printf("%d ", arr[i]);
    free(arr);
    return 0;
}
```

Q10. Memory leak ka example dikhao aur usko free() se fix karo.

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int*)malloc(100 * sizeof(int));
    // ... bahut saara code

    // Memory leak yahan hota hai agar free nahi karte
    free(ptr);      // fix
    ptr = NULL;

    printf("Memory leak avoid kar diya gaya!");
    return 0;
}
```

✅ MIXED PRACTICAL QUESTIONS

Q1. User se n numbers le, dynamic array banao, sum aur average print karo (Dynamic Memory + Array + Loop).

C

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, sum = 0;
    printf("Kitne numbers chahiye? ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));
    if(arr == NULL) {
        printf("Memory nahi mili!");
        return 1;
    }

    for(int i = 0; i < n; i++) {
        printf("Number %d: ", i+1);
        scanf("%d", &arr[i]);
        sum += arr[i];
    }

    printf("Sum = %d\nAverage = %.2f", sum, (float)sum / n);
    free(arr);
    return 0;
}
```

Q2. Structure banao Student ke liye (name, roll, marks). 3 students ka data input lo, array of structure use karke sabse zyada marks wale student ka naam print karo.

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
```

C Language
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
    struct Student s[3];
    int max_idx = 0;

    for(int i = 0; i < 3; i++) {
        printf("\nStudent %d - Name: ", i+1);
        scanf("%s", s[i].name);
        printf("Roll: ");
        scanf("%d", &s[i].roll);
        printf("Marks: ");
        scanf("%f", &s[i].marks);

        if(s[i].marks > s[max_idx].marks)
            max_idx = i;
    }

    printf("\nSabse zyada marks wala student: %s (%.2f)", s[max_idx].name,
s[max_idx].marks);
    return 0;
}
```

Q3. Pointer use karke string reverse karo aur check karo ki palindrome hai ya nahi (String + Pointer + Function).

C

```
#include <stdio.h>
#include <string.h>

void reverseString(char *str) {
    int len = strlen(str);
    for(int i = 0; i < len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}

int main() {
    char str[100];
    printf("String daalo: ");
    scanf("%s", str);

    char original[100];
    strcpy(original, str);
```

```
    reverseString(str);

    printf("Reversed: %s\n", str);
    if(strcmp(original, str) == 0)
        printf("Palindrome hai");
    else
        printf("Palindrome nahi hai");
    return 0;
}
```

Learn C programming from basics to advanced with real-world examples

Q4. Recursion + Pointer use karke factorial calculate karo aur result ko file mein save karo.

C

```
#include <stdio.h>

void factorial(int n, long long *fact) {
    if(n == 0 || n == 1) {
        *fact = 1;
        return;
    }
    factorial(n-1, fact);
    *fact = *fact * n;
}

int main() {
    int n;
    long long fact = 1;
    printf("Number daalo: ");
    scanf("%d", &n);

    factorial(n, &fact);

    FILE *fp = fopen("fact.txt", "w");
    fprintf(fp, "Factorial of %d = %lld", n, fact);
    fclose(fp);

    printf("Factorial = %lld (file mein save ho gaya)", fact);
    return 0;
}
```

Q5. Dynamic 2D array banao, user se values lo aur matrix ka sum print karo.

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int r, c;
    printf("Rows aur Columns daalo: ");
    scanf("%d %d", &r, &c);

    int **mat = (int**)malloc(r * sizeof(int*));
    for(int i = 0; i < r; i++)
        mat[i] = (int*)malloc(c * sizeof(int));

    int sum = 0;
    for(int i = 0; i < r; i++) {
        for(int j = 0; j < c; j++) {
            printf("Element [%d][%d]: ", i, j);
            scanf("%d", &mat[i][j]);
            sum += mat[i][j];
        }
    }

    printf("Total Sum of Matrix = %d", sum);

    for(int i = 0; i < r; i++) free(mat[i]);
    free(mat);
    return 0;
}
```

Q6. Structure + File I/O: Student ka record file mein save karo aur baad mein read karke print karo.

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s;
    printf("Name: ");
    scanf("%s", s.name);
```

THE ARTIFICIAL EDUCATION

```
    printf("Roll: ");
    scanf("%d", &s.roll);
    printf("Marks: ");
    scanf("%f", &s.marks);

    FILE *fp = fopen("students.dat", "wb");
    fwrite(&s, sizeof(s), 1, fp);
    fclose(fp);

    fp = fopen("students.dat", "rb");
    fread(&s, sizeof(s), 1, fp);
    fclose(fp);

    printf("Saved Student: %s | Roll: %d | Marks: %.2f", s.name, s.roll,
s.marks);
    return 0;
}
```

Q7. Array of Structures + Pointer: 3 students ka data input lo aur pointer se sabse zyada marks wale student ko print karo.

C

```
#include <stdio.h>

struct Student {
    char name[50];
    float marks;
};

int main() {
    struct Student s[3];
    struct Student *ptr = s;
    int max_idx = 0;

    for(int i = 0; i < 3; i++) {
        printf("Student %d name: ", i+1);
        scanf("%s", (ptr+i)->name);
        printf("Marks: ");
        scanf("%f", &(ptr+i)->marks);

        if((ptr+i)->marks > (ptr+max_idx)->marks)
            max_idx = i;
    }
```

```
    printf("Topper: %s (%.2f marks)", (ptr+max_idx)->name, (ptr+max_idx)->marks);
    return 0;
}
```

Q8. Recursion + Dynamic Memory: n ka factorial calculate karo aur result dynamic array mein store karo.

C

```c
#include <stdio.h>
#include <stdlib.h>

long long fact(int n) {
    if(n == 0 || n == 1) return 1;
    return n * fact(n-1);
}

int main() {
    int n;
    printf("Number daalo: ");
    scanf("%d", &n);

    long long *result = (long long*)malloc(sizeof(long long));
    *result = fact(n);

    printf("Factorial = %lld", *result);
    free(result);
    return 0;
}
```

Q9. String + Dynamic Memory + Function: User se string le, reverse karo aur nai file mein save karo.

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void reverseStr(char *str) {
    int len = strlen(str);
    for(int i = 0; i < len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}
```

```c
int main() {
    char *str = (char*)malloc(100 * sizeof(char));
    printf("String daalo: ");
    scanf("%s", str);

    reverseStr(str);

    FILE *fp = fopen("reversed.txt", "w");
    fprintf(fp, "%s", str);
    fclose(fp);

    printf("Reversed string file mein save ho gaya!");
    free(str);
    return 0;
}
```

Q10. Menu Driven Program using Structure, Array, File I/O and Conditional (Add/Show/Search Student).
C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s[10];
    int count = 0, choice, r;

    while(1) {
        printf("\n1. Add Student\n2. Show All\n3. Search by Roll\n4.
Exit\nChoice: ");
        scanf("%d", &choice);

        if(choice == 4) break;

        if(choice == 1 && count < 10) {
            printf("Name: ");
            scanf("%s", s[count].name);
            printf("Roll: ");
```

```
            scanf("%d", &s[count].roll);
            printf("Marks: ");
            scanf("%f", &s[count].marks);
            count++;
        }
        else if(choice == 2) {
            for(int i = 0; i < count; i++)
                printf("%s | Roll:%d | Marks:%.2f\n", s[i].name, s[i].roll,
s[i].marks);
        }
        else if(choice == 3) {
            printf("Roll Number: ");
            scanf("%d", &r);
            for(int i = 0; i < count; i++) {
                if(s[i].roll == r) {
                    printf("Found: %s | Marks: %.2f\n", s[i].name, s[i].marks);
                    break;
                }
            }
        }
    }
    return 0;
}
```

Q11. Pointer + Array + Function: Array ka maximum element pointer ke through function mein nikaalo aur print karo.

C

```
#include <stdio.h>

int findMax(int *arr, int size) {
    int max = *arr;
    for(int i = 1; i < size; i++) {
        if(*(arr + i) > max)
            max = *(arr + i);
    }
    return max;
}

int main() {
    int n;
    printf("Kitne numbers? ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));
```

```
    for(int i = 0; i < n; i++) {
        printf("Number %d: ", i+1);
        scanf("%d", &arr[i]);
    }

    printf("Maximum element = %d", findMax(arr, n));
    free(arr);
    return 0;
}
```

Sample Output: Maximum element = 89

Explanation: Pointer arithmetic + function mein array pass kiya gaya.

Q12. Dynamic Memory + Structure: Student ka dynamic array banao aur marks ke hisaab se sort karo (Bubble Sort).

C

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    char name[50];
    float marks;
};

int main() {
    int n;
    printf("Kitne students? ");
    scanf("%d", &n);

    struct Student *s = (struct Student*)malloc(n * sizeof(struct Student));

    for(int i = 0; i < n; i++) {
        printf("Student %d Name: ", i+1);
        scanf("%s", s[i].name);
        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }

    // Bubble Sort by marks
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(s[j].marks < s[j+1].marks) {
                struct Student temp = s[j];
```

```
                s[j] = s[j+1];
                s[j+1] = temp;
            }
        }
    }

    printf("\nSorted by Marks:\n");
    for(int i = 0; i < n; i++)
        printf("%s - %.2f\n", s[i].name, s[i].marks);

    free(s);
    return 0;
}
```

Q13. String + Loop + Conditional: String mein vowels aur consonants count karo aur result file mein save karo.

C

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int vowel = 0, cons = 0;

    printf("String daalo: ");
    fgets(str, 100, stdin);

    for(int i = 0; str[i] != '\0'; i++) {
        char ch = str[i];
        if((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
               ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
                vowel++;
            else
                cons++;
        }
    }

    FILE *fp = fopen("count.txt", "w");
    fprintf(fp, "Vowels = %d\nConsonants = %d", vowel, cons);
    fclose(fp);
```

```
    printf("Vowels = %d, Consonants = %d (file mein save ho gaya)", vowel, cons);
    return 0;
}
```

Q14. Recursion + Pointer + Dynamic Memory: Fibonacci series ko pointer array mein store karo aur print karo.

C

```
#include <stdio.h>
#include <stdlib.h>

void fibSeries(int n, long long *arr) {
    if(n <= 0) return;
    arr[0] = 0;
    if(n == 1) return;
    arr[1] = 1;
    for(int i = 2; i < n; i++)
        arr[i] = arr[i-1] + arr[i-2];
}

int main() {
    int n;
    printf("Kitne terms? ");
    scanf("%d", &n);

    long long *fib = (long long*)malloc(n * sizeof(long long));
    fibSeries(n, fib);

    printf("Fibonacci Series: ");
    for(int i = 0; i < n; i++)
        printf("%lld ", fib[i]);

    free(fib);
    return 0;
}
```

Q15. File I/O + Structure + Array: Multiple students ka data file mein save karo aur unka average marks calculate karo.

C

```
#include <stdio.h>

struct Student {
    char name[50];
```

```
    float marks;
};

int main() {
    int n;
    printf("Kitne students? ");
    scanf("%d", &n);

    struct Student s[10];
    float total = 0;

    FILE *fp = fopen("students.dat", "wb");
    for(int i = 0; i < n; i++) {
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("Marks: ");
        scanf("%f", &s[i].marks);
        total += s[i].marks;
        fwrite(&s[i], sizeof(s[i]), 1, fp);
    }
    fclose(fp);

    printf("Average Marks = %.2f", total / n);
    return 0;
}
```

Q16. Conditional + Loop + Pointer: User se number lo, agar prime hai toh uska reverse pointer ke through print karo.

C

```
#include <stdio.h>

int isPrime(int n) {
    if(n <= 1) return 0;
    for(int i = 2; i * i <= n; i++)
        if(n % i == 0) return 0;
    return 1;
}

void reverseNum(int *num) {
    int rev = 0, temp = *num;
    while(temp > 0) {
        rev = rev * 10 + temp % 10;
```

THE ARTIFICIAL EDUCATION

```
        temp /= 10;
    }
    *num = rev;
}

int main() {
    int num;
    printf("Number daalo: ");
    scanf("%d", &num);

    if(isPrime(num)) {
        reverseNum(&num);
        printf("Prime hai aur Reverse = %d", num);
    } else {
        printf("Prime nahi hai");
    }
    return 0;
}
```

Q17. String + Dynamic Memory + File I/O: User se string le, usko uppercase mein convert karke file mein save karo.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *str = (char*)malloc(200 * sizeof(char));
    printf("String daalo: ");
    fgets(str, 200, stdin);

    for(int i = 0; str[i] != '\0'; i++) {
        if(str[i] >= 'a' && str[i] <= 'z')
            str[i] = str[i] - 32;
    }

    FILE *fp = fopen("upper.txt", "w");
    fputs(str, fp);
    fclose(fp);

    printf("Uppercase string file mein save ho gaya!");
    free(str);
```

```
    return 0;
}
```

Q18. Structure + Recursion + Pointer: Student ka marks structure mein store karo aur recursion se total marks calculate karo.

C

```c
#include <stdio.h>

struct Student {
    char name[50];
    int *marks;
    int sub;
};

int totalMarks(int *marks, int n, int i) {
    if(i == n) return 0;
    return marks[i] + totalMarks(marks, n, i+1);
}

int main() {
    struct Student s;
    printf("Name: ");
    scanf("%s", s.name);
    printf("Subjects: ");
    scanf("%d", &s.sub);

    s.marks = (int*)malloc(s.sub * sizeof(int));
    for(int i = 0; i < s.sub; i++) {
        printf("Subject %d marks: ", i+1);
        scanf("%d", &s.marks[i]);
    }

    printf("Total Marks of %s = %d", s.name, totalMarks(s.marks, s.sub, 0));
    free(s.marks);
    return 0;
}
```

Q19. Array + File I/O + Conditional: 1D array mein numbers lo, even numbers ko ek file mein aur odd ko dusri file mein save karo.

C

```c
#include <stdio.h>
```

```c
int main() {
    int n;
    printf("Kitne numbers? ");
    scanf("%d", &n);

    int arr[100];
    for(int i = 0; i < n; i++) {
        printf("Number %d: ", i+1);
        scanf("%d", &arr[i]);
    }

    FILE *even = fopen("even.txt", "w");
    FILE *odd = fopen("odd.txt", "w");

    for(int i = 0; i < n; i++) {
        if(arr[i] % 2 == 0)
            fprintf(even, "%d\n", arr[i]);
        else
            fprintf(odd, "%d\n", arr[i]);
    }

    fclose(even);
    fclose(odd);
    printf("Even aur Odd files mein save ho gaye!");
    return 0;
}
```

Q20. Pointer + String + Structure: Student ka naam pointer ke through reverse karke structure mein store karo.

C

```c
#include <stdio.h>
#include <string.h>

struct Student {
    char name[50];
};

void reverseName(char *str) {
    int len = strlen(str);
    for(int i = 0; i < len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
```

THE ARTIFICIAL EDUCATION

```
        str[len-1-i] = temp;
    }
}

int main() {
    struct Student s;
    printf("Name daalo: ");
    scanf("%s", s.name);

    reverseName(s.name);
    printf("Reversed Name in Structure: %s", s.name);
    return 0;
}
```

Q21. Dynamic Memory + Loop + Operator: User se n numbers lo, unka sum, average aur maximum print karo.

C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, sum = 0, max = -999999;
    printf("Kitne numbers? ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
        if(arr[i] > max) max = arr[i];
    }

    printf("Sum = %d\nAverage = %.2f\nMaximum = %d", sum, (float)sum/n, max);
    free(arr);
    return 0;
}
```

Q22. Conditional + Function + File I/O: User se number lo, agar prime hai toh uska factorial file mein save karo.

C

```
#include <stdio.h>
```

```c
int isPrime(int n) {
    if(n <= 1) return 0;
    for(int i = 2; i * i <= n; i++)
        if(n % i == 0) return 0;
    return 1;
}

long long fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n-1);
}

int main() {
    int n;
    printf("Number daalo: ");
    scanf("%d", &n);

    if(isPrime(n)) {
        FILE *fp = fopen("fact.txt", "w");
        fprintf(fp, "Factorial of %d = %lld", n, fact(n));
        fclose(fp);
        printf("Prime hai! Factorial file mein save ho gaya.");
    } else {
        printf("Prime nahi hai.");
    }
    return 0;
}
```

Q23. 2D Dynamic Array + Structure + Pointer: Matrix ka addition karo aur result structure mein store karke print karo.

C

```c
#include <stdio.h>
#include <stdlib.h>

struct Matrix {
    int **data;
    int row, col;
};

int main() {
    int r = 2, c = 2;
    struct Matrix m1, m2, res;
```

THE ARTIFICIAL EDUCATION

```c
    m1.row = m2.row = res.row = r;
    m1.col = m2.col = res.col = c;

    m1.data = (int**)malloc(r * sizeof(int*));
    m2.data = (int**)malloc(r * sizeof(int*));
    res.data = (int**)malloc(r * sizeof(int*));

    for(int i = 0; i < r; i++) {
        m1.data[i] = (int*)malloc(c * sizeof(int));
        m2.data[i] = (int*)malloc(c * sizeof(int));
        res.data[i] = (int*)malloc(c * sizeof(int));
    }

    // Sample values
    m1.data[0][0] = 1; m1.data[0][1] = 2;
    m1.data[1][0] = 3; m1.data[1][1] = 4;
    m2.data[0][0] = 5; m2.data[0][1] = 6;
    m2.data[1][0] = 7; m2.data[1][1] = 8;

    for(int i = 0; i < r; i++)
        for(int j = 0; j < c; j++)
            res.data[i][j] = m1.data[i][j] + m2.data[i][j];

    printf("Sum Matrix:\n");
    for(int i = 0; i < r; i++) {
        for(int j = 0; j < c; j++)
            printf("%d ", res.data[i][j]);
        printf("\n");
    }

    // free memory
    for(int i = 0; i < r; i++) {
        free(m1.data[i]); free(m2.data[i]); free(res.data[i]);
    }
    free(m1.data); free(m2.data); free(res.data);
    return 0;
}
```

Q24. Menu Driven Program using all topics: Add student (Structure + Dynamic), Show all, Save to file, Load from file.

C

THE ARTIFICIAL EDUCATION

```c
#include <stdio.h>
#include <stdlib.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student *s = NULL;
    int count = 0, choice;

    while(1) {
        printf("\n1. Add Student\n2. Show All\n3. Save to File\n4. Load from
File\n5. Exit\nChoice: ");
        scanf("%d", &choice);

        if(choice == 5) break;

        if(choice == 1) {
            s = (struct Student*)realloc(s, (count+1) * sizeof(struct Student));
            printf("Name: ");
            scanf("%s", s[count].name);
            printf("Roll: ");
            scanf("%d", &s[count].roll);
            printf("Marks: ");
            scanf("%f", &s[count].marks);
            count++;
        }
        else if(choice == 2) {
            for(int i = 0; i < count; i++)
                printf("%s | Roll:%d | Marks:%.2f\n", s[i].name, s[i].roll,
s[i].marks);
        }
        else if(choice == 3) {
            FILE *fp = fopen("students.dat", "wb");
            fwrite(s, sizeof(struct Student), count, fp);
            fclose(fp);
            printf("Saved to file!");
        }
        else if(choice == 4) {
            FILE *fp = fopen("students.dat", "rb");
```

```
            fseek(fp, 0, SEEK_END);
            long size = ftell(fp);
            count = size / sizeof(struct Student);
            s = (struct Student*)realloc(s, count * sizeof(struct Student));
            rewind(fp);
            fread(s, sizeof(struct Student), count, fp);
            fclose(fp);
            printf("Loaded %d students!", count);
        }
    }
    free(s);
    return 0;
}
```

Q25. Final Mixed Challenge: Recursion + Pointer + File I/O + Structure: Student ka naam reverse karke file mein save karo aur uska factorial (roll number ka) bhi save karo.

C

```
#include <stdio.h>
#include <string.h>

struct Student {
    char name[50];
    int roll;
};

void reverseName(char *str) {
    int len = strlen(str);
    for(int i = 0; i < len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}

long long fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n-1);
}

int main() {
    struct Student s;
    printf("Name: ");
```

```c
    scanf("%s", s.name);
    printf("Roll: ");
    scanf("%d", &s.roll);

    reverseName(s.name);

    FILE *fp = fopen("final.txt", "w");
    fprintf(fp, "Reversed Name: %s\n", s.name);
    fprintf(fp, "Roll Factorial: %lld", fact(s.roll));
    fclose(fp);

    printf("Sab kuch final.txt mein save ho gaya!");
    return 0;
}
```


Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION